$$x_{ij} \in \{0,1\}, \qquad \forall i,j \in V \tag{7.19}$$

约束 (7.18) 可以起到消除子环路的作用。但是由于点集 V 的元素个数为 $2\sim N-1$ 的子集非常多，有 2^N-N-1 个，呈指数增长。如果要在建模一开始就直接将 (7.18) 全部列举出来，是非常耗时的，也许列举该约束的时间都要比求解 TSP 本身的时间长。

为了解决这个问题，求解器（Gurobi 和 CPLEX）提供了一种特殊的添加约束的方法，叫作惰性约束（lazy constraints），这种约束可以通过 callback 实现。惰性约束就是不在一开始构建模型时就将约束加入，而是先加入一部分约束之后就尝试求解模型并获得求解的结果，根据结果判断其是否违背了惰性约束，如果违背了，则将该条约束以 lazy 的方式添加到模型中。再次尝试求解，直到求解的结果不违背任意一条惰性约束为止。容易得知，惰性约束是在算法迭代的过程中，动态地在分支定界算法的分支节点处加入的为了剔除不可行解的约束（消除子环路就是一种剔除不可行解的方式）。

callback 的功能非常强大，我们可以通过 callback 函数控制求解器的求解进程。即首先识别分支定界树中节点的解是否满足特定的条件，如果满足，我们就可以创建相应的约束，这些约束可以通过惰性约束的形式构建并被添加到当前正在求解的模型中。只要我们将实现惰性约束的 callback 函数的函数名作为参数传给模型求解函数 optimize()，Gurobi 就可以自动识别 callback，并调用 callback 函数，然后按照用户的要求在求解过程中动态识别违反约束的情况，并将被违反的约束以 lazy 的形式动态添加到正在求解的模型中。这种方法在分支切割算法、benders 分解及行生成算法（row generation）中用得比较多，想要进阶的读者一般都需要掌握该部分内容。

使用 callback 的步骤（仅针对本问题，其他问题中 callback 的使用方法与该问题类似）

- **第一步**：利用 Gurobi 构建数学模型，只加入前两组约束 (7.16) 和 (7.17)；
- **第二步**：构建一个用来识别当前解中的子环路，并且根据该子环路构建相应的消除子环路的约束表达式的函数 `subtourelim(model,where)`（注意，这个函数的参数 `model`，`where` 是求解器规定好的）。这个函数用于拿到整数规划分支定界迭代过程中当前节点的解的信息，并根据当前节点的解，识别该解中是否存在子环路，如过存在则将消除子环路的约束返回给正在求解的模型，并将其添加到当前模型中；如果不存在子环路，则不进行任何干扰当前模型求解的操作；
- **第三步**：设置当前模型参数 `model.Params.lazyConstraints` 的值为 1，即使用 lazyConstraints。之后，就可以将添加 lazyConstraints 的 callback 函数作为参数，启动算法，求解模型，即 `model.optimize(subtourelim)`。这里，求解器要求必须要将 callback 函数 `subtourelim(model,where)` 作为参数传给 `optimize()` 函数。

具体代码如下。

lazyConstraints.py

```
model._vars = X      # 设置model的外部变量为TSP的决策变量字典X
model.Params.lazyConstraints = 1     # 设置使用lazyConstraints
model.optimize(subtourelim)          # use callback function when executing
        # branch and bound algorithm
```

接下来给出 Python 调用 Gurobi 实现 callback 添加惰性约束的方式求解 TSP 的完整代码。

首先，我们定义以下四个函数，来完成子环路的识别和 callback 添加 lazy Constraints 的实现。

- subtourelim(model, where)：callback 函数，用于为 model 对象动态添加 subtour-elimination 约束；
- computeDegree(graph)：给定一个 graph（二维数组形式，即给定一个邻接矩阵），计算出每个节点的 degree（degree= 每个节点被进入次数 + 被离开的次数）；
- findEdges(graph)：给定一个 graph（二维数组形式，即给定一个邻接矩阵），找到该图中所有的边，例如 $[(1,2),(2,4),(2,5)]$；
- subtour(graph)：给定一个 graph（二维数组形式，即给定一个邻接矩阵），找到该图中包含节点数目最少的子环路，例如 $[2,3,5]$。

以上函数的完整代码如下。其中，函数 subtourelim(model,where) 中调用了函数 computeDegree(graph)、findEdges(graph) 和 subtour(graph)。

TSP Callback lazyConstraints.py

```
# Callback - use lazy constraints to eliminate sub-tours

def subtourelim(model, where):
    if (where == GRB.Callback.MIPSOL):
        # make a list of edges selected in the solution
        print('model._vars', model._vars)
        #         vals = model.cbGetSolution(model._vars)
        x_value = np.zeros([nodeNum + 1, nodeNum + 1])
        for m in model.getVars():
            if (m.varName.startswith('x')):
                #                 print(var[i].varName)
                #                 print(var[i].varName.split('_'))
                a = (int)(m.varName.split('_')[1])
```

```
                b = (int)(m.varName.split('_')[2])
                x_value[a][b] = model.cbGetSolution(m)
        print("solution = ", x_value)
        # find the shortest cycle in the selected edge list
        tour = subtour(x_value)
        print('tour = ', tour)
        if (len(tour) < nodeNum + 1):
            # add subtour elimination constraint for every pair of cities in
            # tour
            print("---add sub tour elimination constraint--")
            for i, j in itertools.combinations(tour, 2):
                print(i, j)

            model.cbLazy(quicksum(model._vars[i, j]
                                  for i, j in itertools.permutations(tour,
    2))
                         <= len(tour) - 1)
            LinExpr = quicksum(model._vars[i, j]
                               for i, j in itertools.permutations(tour, 2))
            print('LinExpr = ', LinExpr)
            print('RHS = ', len(tour) - 1)

        # compute the degree of each node in given graph

def computeDegree(graph):
    degree = np.zeros(len(graph))
    for i in range(len(graph)):
        for j in range(len(graph)):
            if (graph[i][j] > 0.5):
                degree[i] = degree[i] + 1
                degree[j] = degree[j] + 1
    print('degree', degree)
    return degree

# given a graph, get the edges of this graph
def findEdges(graph):
    edges = []
    for i in range(1, len(graph)):
        for j in range(1, len(graph)):
            if (graph[i][j] > 0.5):
                edges.append((i, j))

```

```
    return edges

# Given a tuplelist of edges, find the shortest subtour
def subtour(graph):
    # compute degree of each node
    degree = computeDegree(graph)
    unvisited = []
    for i in range(1, len(degree)):
        if (degree[i] >= 2):
            unvisited.append(i)
    cycle = range(0, nodeNum + 1)  # initial length has 1 more city

    edges = findEdges(graph)
    edges = tuplelist(edges)
    print(edges)
    while unvisited:  # true if list is non-empty
        thiscycle = []
        neighbors = unvisited
        while neighbors:  # true if neighbors is non-empty
            current = neighbors[0]
            thiscycle.append(current)
            unvisited.remove(current)
            neighbors = [j for i, j in edges.select(current, '*') if j in
        unvisited]
            neighbors2 = [i for i, j in edges.select('*', current) if i in
        unvisited]
            if (neighbors2):
                neighbors.extend(neighbors2)

        isLink = ((thiscycle[0], thiscycle[-1]) in edges) or ((thiscycle
        [-1], thiscycle[0]) in edges)   #  注意这里需要考虑[(1,2),(2,1)]这两种
        # 情况
        if (len(cycle) > len(thiscycle) and len(thiscycle) >= 3 and isLink):
            cycle = thiscycle
            return cycle
    return cycle
```

完成上述准备之后，我们调用上述函数，读取数据并建模求解。完整代码如下：

Solve TSP using callback .py

```
# Solve TSP using callback
nodeNum = 10
path = 'c101.txt'
```

```
cost = readData(path, nodeNum)
printData(cost)

model = Model('TSP')

# creat decision variables
X = {}
mu = {}
for i in range(nodeNum + 1):
    mu[i] = model.addVar(lb=0.0
                         , ub=100  # GRB.INFINITY
                         # , obj = distance_initial
                         , vtype=GRB.CONTINUOUS
                         , name="mu_" + str(i)
                         )

    for j in range(nodeNum + 1):
        if (i != j):
            X[i, j] = model.addVar(vtype=GRB.BINARY
                                   , name='x_' + str(i) + '_' + str(j)
                                   )
# set objective function
obj = LinExpr(0)
for key in X.keys():
    i = key[0]
    j = key[1]
    if (i < nodeNum and j < nodeNum):
        obj.addTerms(cost[key[0]][key[1]], X[key])
    elif (i == nodeNum):
        obj.addTerms(cost[0][key[1]], X[key])
    elif (j == nodeNum):
        obj.addTerms(cost[key[0]][0], X[key])

model.setObjective(obj, GRB.MINIMIZE)

# add constraints 1
for j in range(1, nodeNum + 1):
    lhs = LinExpr(0)
    for i in range(0, nodeNum):
        if (i != j):
            lhs.addTerms(1, X[i, j])
    model.addConstr(lhs == 1, name='in_visit_' + str(j))
```

```
46
47 # add constraints 2
48 for i in range(0, nodeNum):
49     lhs = LinExpr(0)
50     for j in range(1, nodeNum + 1):
51         if (i != j):
52             lhs.addTerms(1, X[i, j])
53     model.addConstr(lhs == 1, name='out_visit_' + str(i))
54
55 # add constraints 3
56 for i in range(0,nodeNum):
57     for j in range(i+1,nodeNum):
58         lhs = LinExpr(0)
59         lhs.addTerms(1, X[i,j])
60         lhs.addTerms(1, X[j,i])
61         model.addConstr(lhs <= 1 ,name='avoid_length2_cycle_' + str(i) + '_'
           + str(j))
62
63 # set lazy constraints
64 model._vars = X
65 model.Params.lazyConstraints = 1
66 model.optimize(subtourelim)
67 # subProblem.optimize()
68 x_value = getValue(X, nodeNum)
69 route = getRoute(x_value)
70 print('optimal route:', route)
71 reportMIP(model,route)
72
73 # record the Running time
74 end_time = time.time()
75
76 print('Running time:', round(end_time - start_time, 4), ' seconds')
```

下面用算例 C101 的前 20 个点作为测试算例，求解结果如下：

Solve TSP using callback .py

```
optimal route: [0, 7, 8, 9, 6, 4, 2, 1, 3, 5, 10, 0]
Best MIP Solution:  48.0

Running time: 0.0857  seconds
```

观察 MTZ 建模和 callback 添加惰性这两种方法，后者的运行速度有非常显著的提高。读者可以自行改变算例大小进行测试。

PART THREE

第III部分

运筹优化常用算法及实战

在介绍具体的算法之前，我们首先对本书涉及的算法做一个整体介绍，方便读者厘清各个算法之间的联系。

本书所介绍的算法主要是针对（线性或双线性）混合整数规划问题的精确算法，以最基本的线性规划单纯形算法引入，让初学读者有最基本的感官认知，然后用最短路的经典算法——Dijkstra 算法开启运筹优化算法大门。分支定界算法作为求解混合整数规划的基石算法，我们首先对其进行讲解；割平面算法能够有效提升分支定界的效率，将各种有效不等式构建割平面的算法融合到分支定界算法中，就构成了分支切割算法。这之后我们引入了拉格朗日松弛算法，该算法在运筹学、自动化、计算机等各领域被广泛应用，其原理和实现都比较简单，在运筹优化领域，该算法常常用来提供更好的上界或者下界。然后是能够有效应对较大规模问题的列生成算法，该算法用于求解具有特殊结构的可分解的问题。这之后我们考虑到列生成算法的子问题通常需要设计高效算法求解，所以我们介绍了常用的动态规划算法，作为列生成算法的补充。接下来就是运筹优化算法的进阶算法，包括分支定价算法、Dantzig-Wolfe 分解算法和 Benders 分解算法。这 3 种高阶算法可以应对较为复杂的问题，它们的思路都是将复杂模型拆分为几个简单模型进行迭代求解，这些方法往往可以有效提升问题的求解效率。特别地，Benders 分解算法在两阶段鲁棒优化领域也有不俗表现。

由于本书主要面向初学者和具备一定基础的从业者，加之篇幅有限，所以一些更加复杂的算法并没有涉及。实际上，在本书涉及的这些算法之外还有更加复杂的算法，比如近几年应用比较广的分支定价与切割算法。该算法能够求解更大规模的混合整数规划问题，但本质上还是上述几种算法的综合，只要掌握了分支定价算法和分支切割算法，通过组合，就可以比较顺利地掌握分支定价与切割算法。总之，虽然还有很多本书没有涉及的更加复杂的运筹优化算法，但是在读懂本书之后，想要进行更加深入地学习，是比较容易上手的。

此外，在实际科研和学习中，很多问题都可以用（元）启发式算法（如邻域搜索（Neighborhood Search，NS）、变邻域搜索 (Variable Neighborhood Search，VNS）、大规模自适应邻域搜索（Adaptive Large Neighbor Search，ALNS）、遗传算法（Genetic Algorithm，GA）、禁忌搜索（Tabu Search，TS）、模拟退火（Simulated Annealing，SA）等）快速求解，但本书主要介绍精确算法，所以不涉及这些启发式算法，感兴趣的读者可参考相应的书籍（例如 Burke et al.，2005）。本书涉及算法的整体框架如下图所示。

这些算法之间的联系如下。

（1）单纯形算法是分支定界算法和列生成算法的基础。

（2）分支定界算法 + 割平面算法 → 分支切割算法。

（3）分支定界算法 + 列生成算法 → 分支定价算法。

（4）DW 分解算法常常和列生成算法配合使用。

（5）Benders 分解算法属于行生成算法，也可用于处理随机、鲁棒相关问题。

（6）列生成算法 + 行生成算法 → 列与约束生成算法。

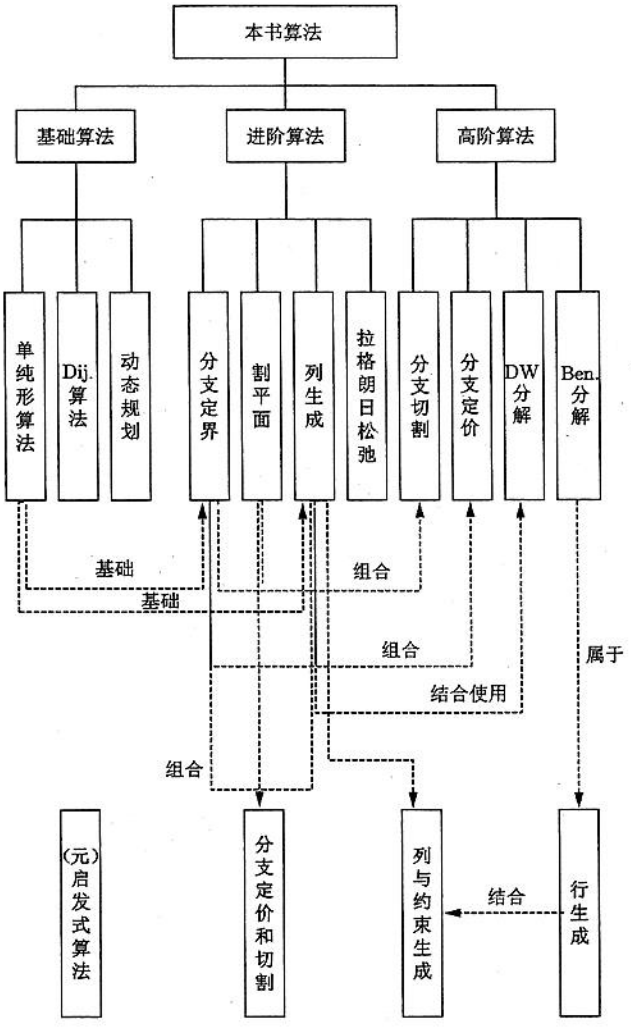
（7）分支定界算法 + 列生成算法 + 割平面算法 → 分支定价与切割算法。

（8）拉格朗日松弛算法常常用来为分支切割、分支定价算法提供较好的上界或者下界。

（9）Dijkstra 算法本质上属于动态规划算法，但是有其局限性。

（10）动态规划算法常常和分支定价算法结合使用，用以高效地求解子问题。

（11）（元）启发式算法用于求解规模较大的问题，可以与精确算法结合使用（比如在精确算法迭代的过程中，快速找到可行解，从而更新上界或者下界，加快算法收敛），也可以单独使用，某些启发式算法还常常用于快速生成初始解（例如 Solomon I1、I2 等）。



本书涉及算法的整体框架

注：Dij. 算法：Dijkstra 算法；DW 分解：Dantzing-Wolfe 分解；Ben. 分解：Benders 分解

如前所述，本书对分支定价与切割算法、列与约束生成算法及启发式算法不做介绍，感兴趣的读者请阅读相关资料自行学习。

第8章　单纯形法

在本章，主要讲述运筹学中常用的算法，包括单纯形法、Dijkstra 算法、分支定界算法、分支切割算法、列生成算法、动态规划算法、分支定价算法、Dantzig-Wolfe 分解算法、Benders 分解算法和拉格朗日松弛算法。

为顺利理解本章内容，需要读者对本科的运筹学课程中线性规划部分有基本的了解。我们先从最基本的单纯形法（Simplex Algorithm）开始。后续章节，例如 Dantzig-Wolfe 分解部分，也用到了这一章的内容。

我们不会像本科运筹学教材那样，从零基础开始讲解单纯形法。假设读者已对单纯形法有了一定的了解，基于此，这里以一个具体的小例子，先来完整地展示一遍单纯形法的迭代过程，然后给出基本的单纯形法的算法伪代码。最后给出了 Python 版本的代码实现。

我们考虑一个目标函数为最大化的线性规划问题，如下：

$$\begin{aligned} \max \quad & \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b} \\ & \boldsymbol{x} \geqslant \boldsymbol{0} \end{aligned} \tag{LP}$$

其中，目标函数系数 $\boldsymbol{c}$、右端常数项（Right Hand Side）$\boldsymbol{b}$ 都是列向量，并且 $\boldsymbol{c} \in \mathbb{R}^{m\times 1}$，$\boldsymbol{x} \in \mathbb{R}^{m\times 1}$，$\boldsymbol{A} \in \mathbb{R}^{m\times n}$，$\boldsymbol{b} \in \mathbb{R}^{m\times 1}$，$m$ 是约束的个数，n 是变量的个数，并且 $m \leqslant n$。本章不考虑退化的情况，我们聚焦于单纯形法的基本步骤。

8.1　线性规划问题的标准形式

在算法迭代之前，首先需要将模型 (LP) 加入松弛变量（Slack Variable）或者剩余变量（Surplus Variable）变成标准形式。所谓标准形式，即满足如下形式：

（1）目标函数为 max 或者 min；

（2）约束条件全部为等于约束；

（3）决策变量取值均为非负；

（4）右端常数均为非负值。

例如，如果约束为

$$x_1 + x_2 \leqslant 4$$

那我们可以加入一个松弛变量 $x_3 \geqslant 0$，将该约束变成

$$x_1 + x_2 + x_3 = 4$$

我们用 x_s 表示松弛变量，则模型 (LP) 就可以写成下面的标准形式：

$$\begin{aligned} \max \quad & \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x}+\boldsymbol{I}\boldsymbol{x}_s=\boldsymbol{b} \\ & \boldsymbol{x},\boldsymbol{x}_s \geqslant \boldsymbol{0} \end{aligned} \qquad \text{(Standard Form)}$$

这个标准形式中，x_s 就可以作为单纯形法的初始可行基，进行算法迭代。

但是很多模型中会有大于或等于约束，例如下面的约束：

$$x_1+x_2 \geqslant 4$$

这种情况下，我们可以加入一个剩余变量 $x_4 \geqslant 0$，将该约束变成

$$x_1+x_2-x_4=4$$

还有一些模型中会出现有些变量无约束的情况。如果一个变量 x 是无约束的，也是可以变为标准形式的。方法是引入两个非负变量 $x',x'' \geqslant 0$，并令 $x=x'-x''$，就可以成功用 x' 和 x'' 将变量 x 完美替换，将原问题转化成标准形式。

对于上面这两种情况，加入剩余变量或者两个辅助变量以后，初始可行基就不易直接观察得出。此时，我们就可以采用加入人工变量的方法将其变成标准形式。比如下面的例子：

$$\begin{aligned} \max \quad & z = x_1+2x_2+3x_3 \\ \text{s.t.} \quad & x_1+x_2+x_3 \leqslant 8 \\ & x_1-x_2+x_3 \geqslant 1 \\ & 2x_1+x_2+2x_3 = 6 \\ & x_1,\ x_2 \geqslant 0 \\ & x_3 \text{ free} \end{aligned}$$

我们为第 1 个约束加入松弛变量，为第 2 个约束加入剩余变量，并且引入 2 个非负变量，处理 x_3 无约束的情况。进行上述操作以后，模型变为

$$\begin{aligned} \max \quad & z = x_1+2x_2+3x_4-3x_5+0x_6+0x_7 \\ \text{s.t.} \quad & x_1+x_2+x_4-x_5+x_6 = 8 \\ & x_1-x_2+x_4-x_5-x_7 = 1 \\ & 2x_1-x_2+2x_4-2x_5 = 6 \\ & x_1,\ x_2,\ x_4,\ x_5,\ x_6,\ x_7 \geqslant 0 \end{aligned}$$

但是上述标准形式就没有很明显的初始可行基。为了构造初始可行基，我们可以为每个约束都人为添加 1 个人工变量，也就是

$$
\begin{aligned}
\max \quad z = {} & x_1+2x_2+3x_4-3x_5+0x_6+0x_7-Mx_8-Mx_9-Mx_{10} \\
\text{s.t.} \quad & x_1+x_2+x_4-x_5+x_6 \qquad +x_8 \qquad\qquad\quad =8 \\
& x_1-x_2+x_4-x_5 \qquad -x_7 \qquad +x_9 \qquad\quad =1 \\
& 2x_1-x_2+2x_4-2x_5 \qquad\qquad\qquad +x_{10} =6 \\
& x_1,\ x_2,\ x_4,\ x_5,\ x_6,\ x_7,\ x_8,\ x_9,\ x_{10} \geqslant 0
\end{aligned}
$$

其中，x_8, x_9, x_{10} 都是人工变量，并且它们在目标函数中的系数都是 $-M$，这是因为我们不希望让这些变量取值为非零。人工变量的加入，使得我们可以很容易就找到模型的初始可行基，可以继续单纯形法迭代。另外，人工变量还可以作为判断模型是否可行的依据。如果在单纯形法迭代完成（max 问题），也就是所有变量的检验数都非正时，基变量中仍然存在非零的人工变量，则该模型无可行解。

写出模型的标准形式以后，我们就可以开始单纯形法的迭代了。由于本书力求简要介绍原理，所以对于单纯形法的详细理论论证不做过多阐述，而是直接给出单纯形法的算法框图，并以一个小例子展示算法迭代的过程，然后给出算法伪代码和 Python 实现。

8.2 单纯形法流程图及详细案例

对于目标函数为最大化的 max 问题，单纯形法的算法框图如图 8.1 所示（见文献《运筹学》教材编写组，2012）。

值得注意的是，用 θ 表示单纯形算法某一步迭代中最小比值原则中的最小比值，则根据最小比值原则选择出基变量 x_l，并选择检验数为负（$\sigma_j < 0$，针对 min 问题）的 x_j 为入基变量（对于 max 问题，选择 $\sigma_j > 0$ 的变量 x_j 为入基变量），在执行出基和入基操作之后，目标函数的改进量为（详细论证见文献胡运权，郭耀煌，2012）

$$\Delta z = \theta(c_j - \boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\boldsymbol{N}_j) = \theta\sigma_j \tag{8.1}$$

由上式可知，当 $\sigma_j > 0$ 时，执行入基操作，可以使目标函数增大；当 $\sigma_j < 0$ 时，执行入基操作，可以使目标函数减小。这也是为什么选择符合相应条件的变量（min 问题选检验数为负的变量，max 问题选检验数为正的变量）作为入基变量，以及将检验数作为判断是否达到最优解的标准的理论依据。这一点在之后的例子中也得到了验证。

根据上述算法框图，下面以一个小例子，用单纯形表的方法，完整地进行一遍迭代操作。考虑下面的模型：

$$
\begin{aligned}
\max \quad & z = 2x_1+4x_2+3x_3 \\
\text{s.t.} \quad & x_1+3x_2+2x_3 \leqslant 80
\end{aligned}
$$

$$3x_1+4x_2+2x_3 \leqslant 60$$
$$2x_1+ x_2+2x_3 \leqslant 40$$
$$x_1,\quad x_2,\quad x_3 \geqslant 0$$

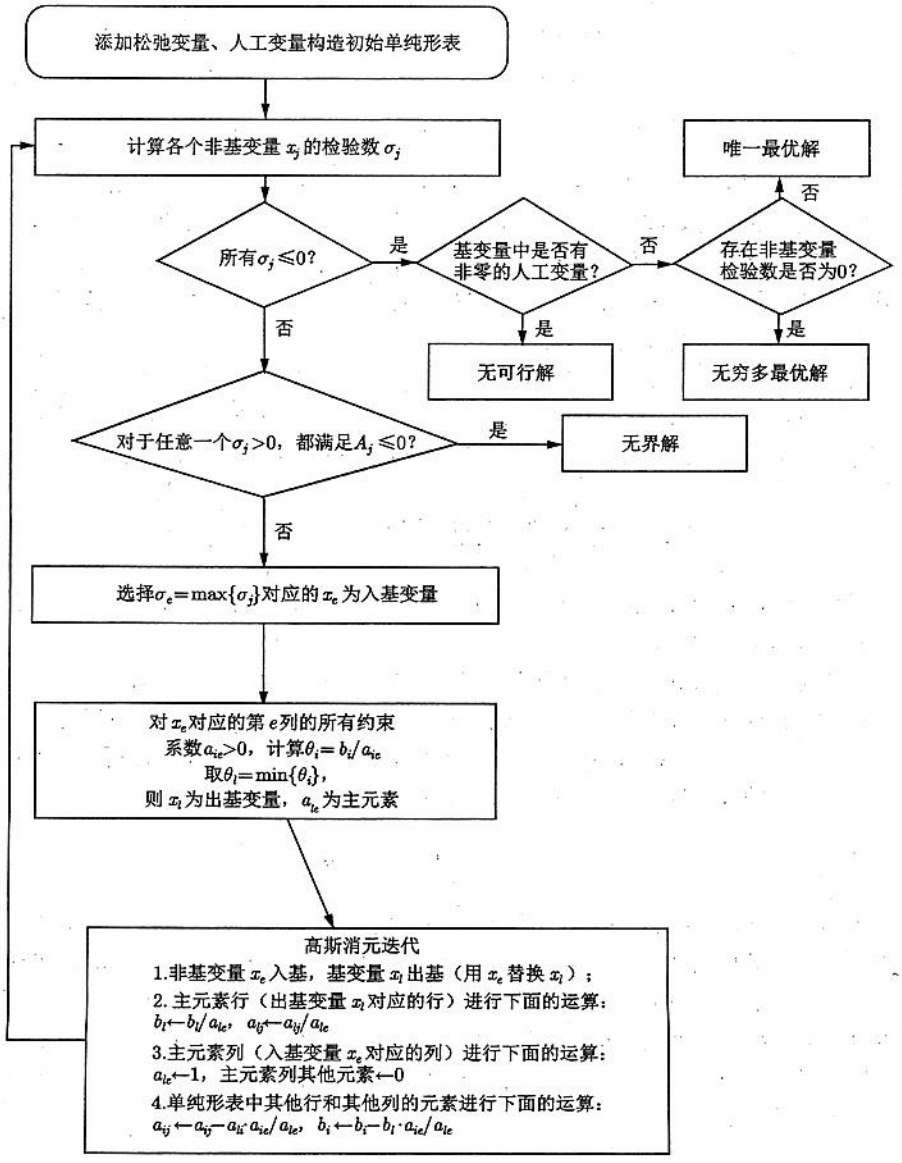


图 8.1　单纯形算法框图

上述问题对应的对偶问题为

$$\begin{aligned}
\min \quad & W = 80y_1 + 60y_2 + 40y_3 \\
\text{s.t.} \quad & y_1 + 3y_2 + 2y_3 \geqslant 2 \\
& 3y_1 + 4y_2 + y_3 \geqslant 4 \\
& 2y_1 + 2y_2 + 2y_3 \geqslant 3 \\
& y_1,\ y_2,\ y_3 \geqslant 0
\end{aligned}$$

加入松弛变量，将其变成如下标准形式：

$$\begin{aligned}
\max \quad & z = 2x_1 + 4x_2 + 3x_3 + 0x_4 + 0x_5 + 0x_6 \\
\text{s.t.} \quad & x_1 + 3x_2 + 2x_3 + x_4 = 80 \\
& 3x_1 + 4x_2 + 2x_3 + x_5 = 60 \\
& 2x_1 + x_2 + 2x_3 + x_6 = 40 \\
& x_1,\ x_2,\ x_3,\ x_4,\ x_5,\ x_6 \geqslant 0
\end{aligned}$$

接下来可以用单纯形表的方法来进行算法迭代。关于单纯形表的形式，这里需要强调一下，国内一些教材中的单纯形表的形式和国外教材的有一些区别。例如按照运筹学编写组的《运筹学》教材中的方法，本算例的初始单纯形表就是如表 8.1 所示的形式。

表 8.1 初始单纯形表：形式 1

c_j			2	4	3	0	0	0	θ_i
c_B	x_B	b	x_1	x_2	x_3	x_4	x_5	x_6	
0	x_4	80	1	3	2	1	0	0	$\frac{80}{3}$
0	x_5	60	3	4	2	0	1	0	15
0	x_6	40	2	1	2	0	0	1	40
$c_j - z_j$			2	4	3	0	0	0	

其中几点说明如下。

（1）最后一行 $c_j - z_j = c_j - \sum_{i=1}^{m} c_i a_{ij} = c_j - \boldsymbol{c}_{\boldsymbol{B}}^{\mathrm{T}}(\boldsymbol{B}^{-1}\boldsymbol{A}_j) = \sigma_j$，表示所有变量对应的列的检验数。

（2）最后一行中，松弛变量 x_4, x_5, x_6 的检验数的相反数，就是该次迭代对应的对偶问题的解；在原问题未达到最优解时，单纯形表中对应的对偶问题的解是非可行解，在最优单纯形表中，如果原问题可行，那么对偶问题的解就是可行的，同时该解也是对偶问题的最优解。

（3）最后一列是最小比值准则（minimum ratio test）的部分。

上面的形式非常直观，但是还有另外一种形式。我们先来看一下上述标准形式的目

标函数

$$z = 2x_1 + 4x_2 + 3x_3$$

该式可以变成

$$z - 2x_1 - 4x_2 - 3x_3 = 0 \tag{8.2}$$

我们将 (8.2) 与模型的 3 个约束放在一起，组成如下 4 个方程：

$$\begin{aligned}
z-2x_1-4x_2-3x_3-0x_4-0x_5-0x_6 &= 0 && \text{(Row 0)}\\
x_1+3x_2+2x_3+x_4 &= 80 && \text{(Row 1)}\\
3x_1+4x_2+2x_3+x_5 &= 60 && \text{(Row 2)}\\
2x_1+x_2+2x_3+x_6 &= 40 && \text{(Row 3)}
\end{aligned}$$

很多英文教材中会把这 4 个方程分别称为 Row 0、Row 1 等。其中 Row 0 对应目标函数行，其余行代表约束。我们将这个方程组变成表格的形式，再将基变量放在第 1 列，就可以写出另外一种形式的单纯形表，如表 8.2 所示。

表 8.2　初始单纯形表：形式 2

Iteration	Basic	Eq.	Coefficient of							RHS	θ_i
			z	x_1	x_2	x_3	x_4	x_5	x_6		
0	z	(0)	1	−2	−4	−3	0	0	0	0	
	x_4	(1)	0	1	3	2	1	0	0	80	$\frac{80}{3}$
	x_5	(2)	0	3	4	2	0	1	0	60	15
	x_6	(3)	0	2	1	2	0	0	1	40	40

其中几点说明如下。

（1）Row 0 行中，变量对应的系数的相反数就是该变量的检验数。例如表 8.2 的 Row 0 中，x_1 的系数为 -2，则 x_1 的检验数为 2。

（2）Row 0 行中，松弛变量对应的系数的值就是对偶变量的取值。例如表 8.2 的 Row 0 中，x_4 的系数为 0，则当前主问题第一个约束对应的对偶变量 y_1 的取值为 0；同理，y_2, y_3 取值也为 0；此时，对偶问题不可行（因为 $y_1 + 3y_2 + 2y_3 = 0 < 2$），但是原问题可行 $(x_1 = x_2 = x_3 = 0, x_4 = 80, x_5 = 60, x_6 = 40)$。通过单纯形法迭代，最终会得到原问题和对偶问题都可行，此时原问题和对偶问题同时取到最优解。

（3）Row 0 行的 RHS 列的数字，就是当前的目标函数 z 的值。

（4）同样地，最后一列是最小比值准则（minimum ratio test）的部分。

我们用第二种形式的单纯形表，将算法完整地迭代一遍。下面直接给出完整步骤，具体细节不再赘述，详细迭代过程如表 8.2～表 8.5 所示。

表 8.3 初始单纯形表

Iteration	Basic	Eq.	Coefficient of							RHS	θ_i
			z	x_1	x_2	x_3	x_4	x_5	x_6		
0	z	(0)	1	−2	−4	−3	0	0	0	0	
	x_4	(1)	0	1	3	2	1	0	0	80	$\frac{80}{3}$
	x_5	(2)	0	3	4	2	0	1	0	60	15
	x_6	(3)	0	2	1	2	0	0	1	40	40

表 8.4 第 1 步迭代

Iteration	Basic	Eq.	Coefficient of							RHS	θ_i
			z	x_1	x_2	x_3	x_4	x_5	x_6		
1	z	(0)	1	1	0	−1	0	1	0	60	
	x_4	(1)	0	−1.25	0	0.5	1	−0.75	0	35	70
	x_2	(2)	0	0.75	1	0.5	0	0.25	0	15	30
	x_6	(3)	0	1.25	0	1.5	0	−0.25	1	25	16.67

经过第 1 步迭代，发现目标函数从 0 增加到 60，变化量正好与之前介绍的结论一致。即 $\Delta z = \theta\sigma_2 = 15 \times 4 = 60$。

表 8.5 第 2 步迭代

Iteration	Basic	Eq.	Coefficient of							RHS	θ_i
			z	x_1	x_2	x_3	x_4	x_5	x_6		
2	z	(0)	1	1.83	0	0	0	0.83	0.67	76.67	
	x_4	(1)	0	−1.67	0	0	1	−0.67	−0.33	26.67	–
	x_2	(2)	0	0.33	1	0	0	0.33	−0.33	6.67	–
	x_3	(3)	0	0.83	0	1	0	−0.17	0.67	16.67	–

迭代之后，目标函数的改进量为 $\Delta z = 76.67 - 60 = 16.67 = \theta\sigma_2 = 16.67 \times 1$。又一次验证了之前介绍的结论。

经过 2 步迭代，Row 0 中所有非基变量的系数均大于 0，说明所有非基变量的检验数均为负数。此时达到了算法停止条件，我们得到了最优解 $z^* = 76.67, x_1 = 0, x_2 = 6.67, x_3 = 16.67$。

从上述迭代中看到，每步迭代中，原问题都是可行的。我们来查看每步迭代相应的对偶问题的情况。

在第 1 步迭代中，$y_1 = y_3 = 0, y_2 = 1$，对偶问题约束 1 和约束 2 成立，但是约束 3 不满足（$2y_1 + 2y_2 + 2y_3 = 0 + 2 + 0 = 2 < 3$），因此对偶问题不可行。

在第 2 步迭代中，$y_1 = 0, y_2 = 0.83, y_3 = 0.67$，对偶问题约束 1：$y_1 + 3y_2 + 2y_3 = 0 + 2.49 + 1.34 = 3.83 > 2$，成立；约束 2：$3y_1 + 4y_2 + y_3 = 0 + 3.32 + 0.67 = 4$（数值精

度问题)；约束 3：$2y_1+2y_2+2y_3=0+1.66+1.34=3$，成立。因此对偶问题可行。此时原问题和对偶问题都可行，原问题和对偶问题均达到最优解。

下面是单纯形法的另一种理解。单纯形法是以一个原问题（Primal）的可行解作为起始（此时对偶问题不可行），逐步迭代，更新原问题的解，迭代过程中对偶问题的解也同时被不断地更新，一直到对偶问题也变得可行，算法终止。此时，原问题和对偶问题同时得到了最优解。

这种单纯形法被称为原始单纯形法（Primal Simplex）。相应地，还有一种对偶单纯形法（Dual Simplex），该方法以一个对偶问题的可行解开始（此时原问题不可行），逐步迭代，直到原问题也可行，算法终止。另外，还有一种衍生的算法，叫作原对偶法（Primal-Dual Algorithm），是根据互补松弛条件（Complementary Slackness Condition）设计的一种算法。原对偶法是从对偶可行解出发，在满足互补松弛条件的前提下，使得原问题变量的取值不断向可行解的方向更新。

定理 8.2.1 *互补松弛条件（Complementary Slackness Condition）如果有线性规划* $\max\{\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}|\boldsymbol{A}\boldsymbol{x}\geqslant\boldsymbol{b},\boldsymbol{x}\geqslant\boldsymbol{0}\}$，$y$ *是其对应的对偶问题的决策变量，则有以下结论。*

（1）*原始互补松弛条件：如果* $x_j>0$，*则* $[\boldsymbol{A}^{\mathrm{T}}\boldsymbol{y}]_j=\boldsymbol{c}_j$；

（2）*对偶互补松弛条件：如果* $y_i>0$，*则* $[\boldsymbol{A}\boldsymbol{x}]_i=\boldsymbol{b}_i$。

根据互补松弛性定理（Complementary Slackness Property），在原问题和对偶问题均达到最优的时候，原问题决策变量和对偶问题对应的松弛变量的乘积为 0，原问题的松弛变量和对偶问题对应的决策变量的乘积也为 0。在原问题和对偶问题不都是最优解时，这个性质并不一定成立。

而 Primal-Dual 算法在迭代的过程中，以互补松弛条件为桥梁，不断地让原问题的取值向着可行解的方向更新，非常巧妙地利用了互补松弛定理。当然，也有一些基于 Primal-Dual 方法设计的近似算法，感兴趣的读者可以阅读相关文献进行深入学习。

8.3 大 M 法和两阶段法

除了最基本的单纯形法之外，还有很多拓展版本的单纯形法。包括大 M 法、两阶段法、对偶单纯形法、改进单纯形法等。我们在这里简要介绍大 M 法与两阶段法，至于其余方法，读者可以参阅其他运筹学书籍。

大 M 法其实和基本单纯形法一致。只是在一些原本就有等式约束的模型中，通过加入人工变量的方法构造初始可行基。但是这些人工变量是为了求解问题方便而额外添加的，我们并不希望这些变量影响到目标函数的取值。因此在 max 问题中，我们将这些人工变量的目标函数系数设置成 $-M$，其中 M 是一个很大的正数。这样，如果原问题要得到最优解，这些人工变量必须要从基变量中替换出。因为如果人工变量是基变量，目标函数就会得到一个很大的负项，一定不可能取到最优。将设置好目标系数后的模型，用 8.2 节中讲述的单纯形表进行迭代，就可以求解原模型。

而两阶段法是一种区别于大 M 法的方法，它也是用于求解加入了人工变量之后的线

性规划问题的。下面考虑之前介绍过的模型：

$$
\begin{aligned}
\max \quad & z = x_1+2x_2+3x_4-3x_5+0x_6+0x_7-Mx_8-Mx_9-Mx_{10} \\
\text{s.t.} \quad & x_1+ x_2+ x_4- x_5+ x_6 \quad + x_8 = 8 \\
& x_1- x_2+ x_4- x_5 \quad - x_7 \quad + x_9 = 1 \\
& 2x_1- x_2+2x_4-2x_5 \quad + x_{10} = 6 \\
& x_1,\ x_2,\ x_4,\ x_5,\ x_6,\ x_7,\ x_8,\ x_9,\ x_{10} \geqslant 0
\end{aligned}
$$

其中，x_8, x_9, x_{10} 都是人工变量。两阶段法分为两个阶段。

第一个阶段：在原问题中添加人工变量，将其变成标准形式。然后构造一个新的线性规划问题。这个问题的约束就是标准形式中的所有约束，但是目标函数中仅包含所有人工变量，并且目标函数为最小化。对于上述模型，我们构造

$$
\begin{aligned}
\min \quad & w = x_8+ x_9+x_{10} \\
\text{s.t.} \quad & x_1+ x_2+ x_4- x_5+ x_6 \quad + x_8 = 8 \\
& x_1- x_2+ x_4- x_5 \quad - x_7 \quad + x_9 = 1 \\
& 2x_1- x_2+2x_4-2x_5 \quad + x_{10} = 6 \\
& x_1,\ x_2,\ x_4,\ x_5,\ x_6,\ x_7,\ x_8,\ x_9,\ x_{10} \geqslant 0
\end{aligned}
$$

用单纯形法求解上述问题，如果得到最优解为 $w = 0$，这说明原问题存在可行解，可以进行下一阶段的迭代；否则原问题不可行，无须进行接下来的计算。

第二个阶段：在第一个阶段迭代得到的最优单纯形表中，删除所有人工变量对应的列。将目标函数行对应的系数全部替换为原问题的目标函数系数，并且问题也改为 max 问题。也就是将目标函数更改为

$$\max \quad z = x_1 + 2x_2 + 3x_4 - 3x_5$$

将更换了目标函数，删除了人工变量之后的单纯形表作为第二阶段的初始单纯形表，重新进行单纯形算法迭代，最终得到最优解。即完成了两阶段法的所有步骤。对两阶段法，在之后的 Dantzig-Wolfe Decomposition 部分还会有所涉及。

8.4 单纯形法伪代码

下面给出单纯形算法的伪代码，方便读者进行代码实现，也方便理解后文提供的代码。

Algorithm 1 单纯形算法

1: 初始化：约束个数 m，变量个数 n, 约束系数矩阵 $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, 成本列向量 $\boldsymbol{c} \in \mathbb{R}^n$, 变量列表 $\boldsymbol{x} \in \mathbb{R}^n$, $\epsilon = 0.001$

2: 添加松弛变量 $\boldsymbol{x}_s$，并将 $\boldsymbol{Ax} \leqslant \boldsymbol{b}$ 转化为标准形 $\boldsymbol{A'x} = \boldsymbol{b}$

设置初始基为 $\boldsymbol{B} \leftarrow \{x_s\}$，其成本向量为 c_B
设置非基变量 $\boldsymbol{NB} \leftarrow \{x\}$，对应的成本向量和约束矩阵 c_N, $\boldsymbol{A}_N$
设置初始 $\boldsymbol{B}^{-1} \leftarrow \boldsymbol{I}$($\boldsymbol{I}$ 表示单位矩阵)
solutionStatus ← NULL
x^* ← NULL
$z^* \leftarrow -\infty$
reducedCost $\leftarrow \boldsymbol{c}_N - \boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\boldsymbol{A}_N$
$\sigma^* \leftarrow \max\{\text{reducedCost}\}$
while $\sigma^* \geqslant \epsilon$ **do**
  for 对每个 j，若满足 $\text{reducedCost}_j > 0$ **do**
    if $A' \leqslant 0$ **then**
      solutionStatus ← UNBOUNDED
      return z^*, x^*, solutionStatus
    end if
  end for
  选择入基变量
  入基变量 $x_e \leftarrow \max$ {reducedCost}
  更新当前基变量和非基变量 $\boldsymbol{B} \leftarrow \boldsymbol{B} \cup \{x_e\}$, $\boldsymbol{NB} \leftarrow \boldsymbol{NB} \backslash \{x_e\}$
  选择出基变量: 最小比值原则
  出基变量 $x_l \leftarrow \min\limits_i \left\{ \dfrac{b_i}{a_i^e} \right\}$
  更新当前基变量和非基变量 $\boldsymbol{B} \leftarrow \boldsymbol{B} \backslash \{x_l\}$, $\boldsymbol{NB} \leftarrow \boldsymbol{NB} \cup \{x_l\}$
  高斯消元
  用高斯消元法更新 $\boldsymbol{A}', \boldsymbol{b}$
  (最终使主元素为 1, 主元所在列的其他系数变为 0)
  用 $\boldsymbol{B}, \boldsymbol{NB}, \boldsymbol{A}'$ 更新 $\boldsymbol{c}_N, \boldsymbol{c}_B, \boldsymbol{A}_N$ 和 $\boldsymbol{B}^{-1}$
  reducedCost $\leftarrow \boldsymbol{c}_N^{\mathrm{T}} - \boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\boldsymbol{A}_N$
  $\sigma^* \leftarrow \max\{\text{reducedCost}\}$
end while
检查解的状态
if 在当前基 B 中存在非零人工变量 **then**
  solutionStatus ← INFEASIBLE
  return z^*, x^*, solutionStatus
else if 非基变量中存在检验数为零的变量 **then**
  solutionStatus ← 多个最优解
end if
solutionStatus ← OPTIMAL

39: $x^* \leftarrow B^{-1}b$

40: $z^* \leftarrow c_B^{\mathrm{T}} B^{-1}b$

41: **return** z^*, x^*, solutionStatus

8.5 Python 实现单纯形法

Python 实现单纯形法代码如下：

Simplex Algorithm

```
#!/usr/bin/env python
# coding: utf-8
'''
@ author :            Liu Xinglu
@ institute :       Tsinghua University
@ date :          2020 年 9 月 27 日
'''

# # Simplex Algorithm


# the maximization problem :
'''
max Z = 2 * x_1 + 3 * x_2
          x_1 + 2 * x_2 <= 8
       4 * x_1 < = 16
                 4 * x_2 <= 12
          x_1, x_2 >= 0


'''

# add slack variables and tranform the problem into standard form
'''
max Z = 2 * x_1 + 3 * x_2
          x_1 + 2 * x_2 + x_3     == 8
       4 * x_1 + x_4 == 16
                 4 * x_2 + x_5 == 12
          x_1, x_2, x_3, x_4, x_ >= 0

'''
import numpy as np
```

```
import pandas as pd
import copy

Basic = [2, 3, 4]
Nonbasic = [0, 1]
c = np.array([2, 3, 0, 0, 0]).astype(float)
c_B = np.array([0, 0, 0]).astype(float)
c_N = np.array([2, 3]).astype(float)
A = np.array([[1, 2, 1, 0, 0]
              , [4, 0, 0, 1, 0]
              , [0, 4, 0, 0, 1]]).astype(float)
A_N = np.array([[1, 2]
                , [4, 0]
                , [0, 4]]).astype(float)
b = np.array([8, 16, 12]).astype(float)
B_inv = np.array([[1, 0, 0]
         , [0, 1, 0]
         , [0, 0, 1]]).astype(float)

x_opt = np.array([0, 0, 0, 0, 0]).astype(float)
z_opt = 0

solutionStatus = None

row_num = len(A)
column_num = len(A[0])

reducedCost = c_N - np.dot(np.dot(c_B, B_inv), A_N)
reducedCost

max_sigma = max(reducedCost)
# print(np.argmax(reducedCost))
eps = 0.001


iterNum = 1
while(max_sigma >= eps):
    # indetify unbounded
    '''
    pass
    '''
    # Determine the entering basic variable
```

```
    enter_var_index =  Nonbasic[np.argmax(reducedCost)]
    print('enter_var_index:', enter_var_index)

    # Determine the leaving basic variable : Minimum ratio test
    min_ratio = 1000000
    leave_var_index = 0
    for i in range(row_num):
        print('b:', b[i], '\t A:', A[i][enter_var_index], '\t ratio:', b[i]/
      A[i][enter_var_index])
        if(A[i][enter_var_index] == 0):
            # solutionStatus = 'Model is infeasible'
             continue
            # return solutionStatus
        elif(b[i]/A[i][enter_var_index] < min_ratio and b[i]/A[i][enter_var_
      index] > 0):
             min_ratio = b[i]/A[i][enter_var_index]
             leave_var_index = i
             # print(min_ratio)

    # process entering basis and leaving basis
    leave_var = Basic[leave_var_index]
    Basic[leave_var_index] = enter_var_index
    Nonbasic.remove(enter_var_index)
    Nonbasic.append(leave_var)
    Nonbasic.sort()

    # Gaussian elimination
    # update pivot row
    pivot_number = A[leave_var_index][enter_var_index]
    print('pivot_number : ', pivot_number)
    for col in range(column_num):
        A[leave_var_index][col] = A[leave_var_index][col]/pivot_number
    b[leave_var_index] = b[leave_var_index] / pivot_number
    # update other rows
    for row in range(row_num):
        if(row != leave_var_index):
            factor = -A[row][enter_var_index] / 1.0
            for col in range(column_num):
                A[row][col] = A[row][col] + factor * A[leave_var_index][col]
            b[row] = b[row] + factor * b[leave_var_index]

    # update c_N, c_B, A_N and B_inv
```

```
    for i in range(len(Nonbasic)):
        var_index = Nonbasic[i]
        c_N[i] = c[var_index]
    for i in range(len(Basic)):
        var_index = Basic[i]
        c_B[i] = c[var_index]
    for i in range(row_num):
        for j in range(len(Nonbasic)):
            var_index = Nonbasic[j]
            A_N[i][j] = A[i][var_index]
    for i in range(len(Basic)):
        col = Basic[i]
        for row in range(row_num):
            B_inv[row][i] = A[row][col]


    # update reduced cost
    reducedCost = c_N - np.dot(np.dot(c_B, B_inv), A_N)
    max_sigma = max(reducedCost)
    iterNum += 1


# check the solution status
for i in range(len(reducedCost)):
    if(reducedCost[i] == 0):
        solution_status = 'Alternative optimal solution'
        break
    else:
        solution_status = 'Optimal'
# get the solution
x_basic = np.dot(B_inv, b)
x_opt = np.array([0.0] * column_num).astype(float)
for i in range(len(Basic)):
    basic_var_index = Basic[i]
    x_opt[basic_var_index] = x_basic[i]
z_opt = np.dot(np.dot(c_B, B_inv), b)

print('Simplex iteration:', iterNum)
print('objective:', z_opt)
print('optimal solution:', x_opt)
```

执行结果如下：

Results

```
Simplex iteration: 4
objective: 14.0
optimal solution: [4. 2. 0. 0. 4.]
```

第9章 Dijkstra算法

9.1 Dijkstra 算法求解最短路问题详解

在图论中，最短路问题（Shortest Path Problem，SPP）是一个非常重要而且基础的问题。该问题在诸多其他运筹学问题中经常涉及。前面的章节探讨了 SPP 的数学模型，但是求解是直接调用求解器进行求解的。其实这个问题并不是 NP-hard 问题。有一些非常好的算法可以在多项式时间内将其求解到最优解。本节将要介绍的 Dijkstra 算法就是其中一个非常著名的算法，该算法由荷兰计算机科学家 Edsger W. Dijkstra 于 1959 年提出（Dijkstra et al., 1959）。关于 Dijkstra 算法，网上也有很多高质量的资料可供参考 ①~③。本章详细介绍该算法的原理、伪代码及代码实现。

我们仍然采取之前的例子，如图 9.1 所示，以点 1 为起点，点 7 为终点，最短路为 $1 \to 2 \to 4 \to 3 \to 6 \to 7$，总距离为 45。

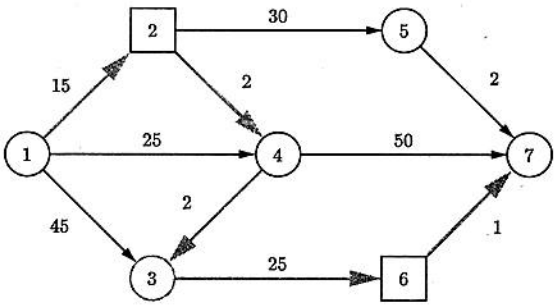


图 9.1 最短路问题：最优解

上述最短路径可以利用 Dijkstra 算法轻松得到，对于规模更大、网络结构更复杂的算例，Dijkstra 算法也可以快速找到最短路径。为了介绍简便，我们引入另一个更简单的网络详细地执行一遍 Dijkstra 算法。图 9.2 所示示例网络，我们定义该图为 G_0。

① https://www.codingame.com/playgrounds/1608/shortest-paths-with-dijkstras-algorithm/dijkstras-algorithm.

② https://brilliant.org/wiki/dijkstras-short-path-finder/.

③ http://www.gitta.info/Accessibiliti/en/html/Dijkstra_learningObject1.html.

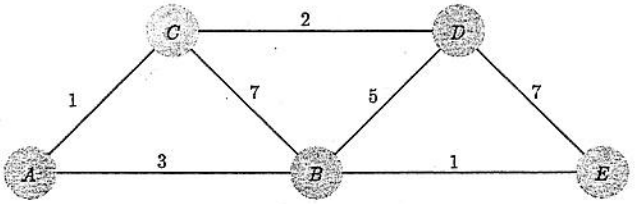


图 9.2 最短路问题：示例网络

我们选定点 C 作为出发点。首先，我们用每一个点到点 C 的最小距离为该点进行标号。对于点 C，我们将初始化距离标号为 0，对于其余的点，我们暂且不知道它们距离点 C 的最短距离是多少，因此姑且用 ∞ 来标号。执行完这一步初始化以后，图 G_0 变为如图 9.3 所示的形式。

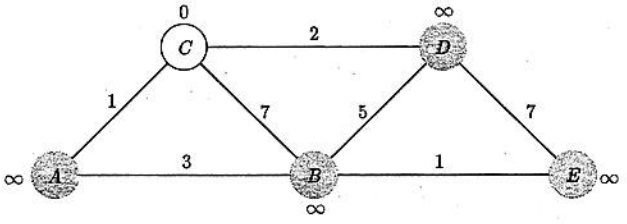


图 9.3 Dijkstra 算法：初始化

为了记录当前位置，用 ○ 来表示当前节点的位置。

然后以任意顺序对当前节点邻接节点依次进行探索。C 的邻接节点有 3 个，它们是 $\{A, B, D\}$，按照这个顺序来检查，首先从 A 开始，将当前节点 C 的最小距离（此时为 0）与当前节点与 A 点连接的边上的权重（为 1）相加，得到 $0+1=1$；然后将该值与 A 点当前的最小距离 ∞ 相比较，然后取 $\{1, \infty\}$ 内的最小值 1，并将点 A 的最小距离更新为 1，如图 9.4 所示。

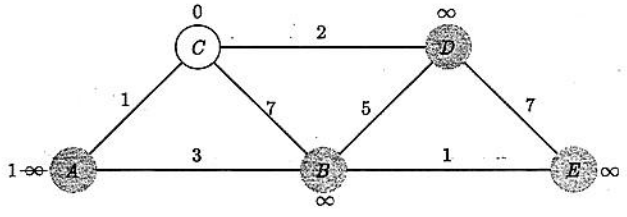


图 9.4 更新节点 A（1）

利用同样的方法更新点 B 的最小距离为 $\min\{0+7, \infty\}=7$；更新点 D 的最小距离为 $\min\{0+2, \infty\}=2$。至此，我们已经探索了点 C 的所有邻接节点。因此，将 C 标记为已探索，用 ✓ 表示，如图 9.5 所示。

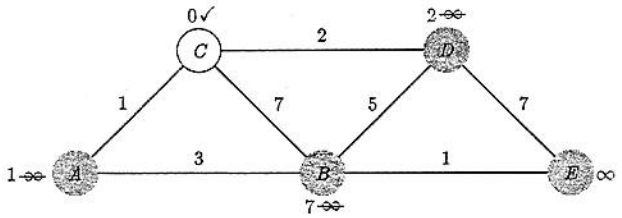


图 9.5 更新节点 B 和 D（2）

现在需要选择一个新的当前点。这个节点必须是所有未被探索的节点中最小距离最小的那个点。此时，未被探索的点集为 $\{A,B,D,E\}$，它们的最小距离的最小值为 $\min\{1,7,2,\infty\}=1$，因此选择点 A 为新的当前点。这里仍用 ○ 标识。对于已经被探索的节点，用 ✓ 来标记，如图 9.6 所示。

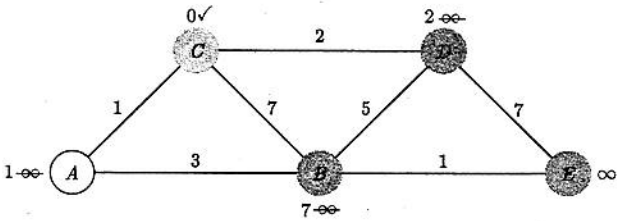


图 9.6 当前节点 A（1）

现在重复该算法。检查当前节点的邻居，而忽略已被探索的节点。这意味着只检查点 B。对于点 B，用该点的当前最短距离 1 加上 A,B 之间连线的权重，得到 $1+3=4$，然后与点 B 的当前最小距离做比较取最小值，得到 $\min\{1+3,7\}=4$，因此将点 B 的最短距离更新为 4。此时，点 A 已经没有其他邻接点，也就意味着已经完成了点 A 的探索，将点 A 标记为已探索，如图 9.7 所示。

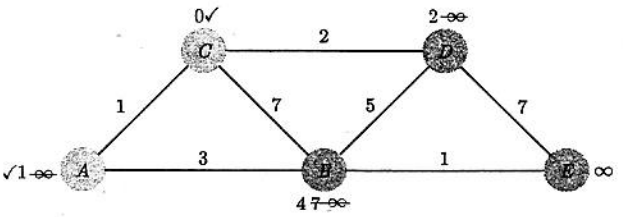


图 9.7 当前节点 A（2）

之后继续进行探索。我们在所有未被探索的点中，选择一个最小距离最小的点，作为新的当前点，未被探索的点集为 $\{B,D,E\}$，它们的最小距离的最小值为 $\min\{4,2,\infty\}=2$，因此选择点 D 为新的当前点，如图 9.8 所示。

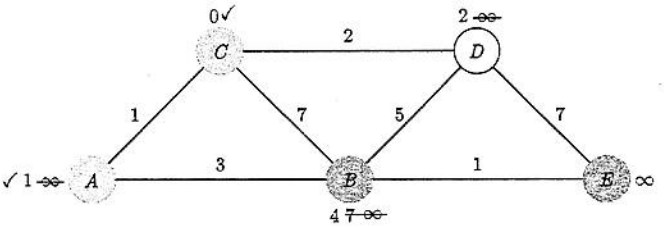


图 9.8 当前节点 D

重复执行算法，D 点的未被探索的邻接节点为 $\{B,E\}$，因此，依次检查 B 和 E。

对于 B，将当前点 D 的当前最短距离 2 加上 D 到 B 的权重，再与点 B 的当前最小距离做比较取最小值，得到 $\min\{2+5,4\}=4$。对点 E，做同样的操作，得到 $\min\{2+7,\infty\}=9$。保持点 B 的最小距离为 4，然后更新点 E 的最小距离为 9。至此，点 D 的所有未探索的邻接节点都已经检查完毕。我们将点 D 标记为已探索。

然后又需要确定新的当前点。在未探索的所有节点中，选择最小距离最小的点 B 作为新的当前点，如图 9.9 所示。

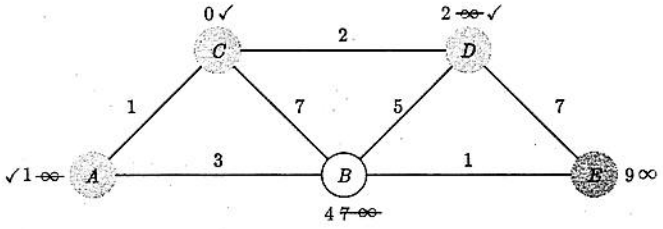


图 9.9 当前节点 B

这里，除了点 B 以外，剩余的未被探索的点只有 E，只需要检查点 E，还是执行同样的操作 $\min\{4+1,9\}=5$，因此将点 E 的最短距离更新为 5。然后点 B 的未被探索的邻接节点也已被检查完毕。标记点 B 为已探索，设置点 E 为新的当前节点，如图 9.10 所示。

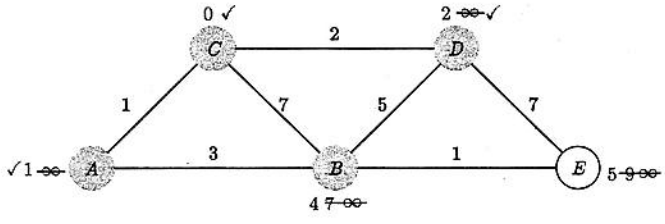


图 9.10 当前节点 E

点 E 没有任何未被探索的邻接点，因此不用做任何检查操作，直接将点 E 标记为已探索。

由于没有未被探索的节点，所以算法执行结束了。现在，每个节点的最小距离实际上表示该节点到出发节点 C 之间的最小距离。

从而得到了从起始点 C 到达所有其他节点 $\{A, B, D, E\}$ 的最小距离。但是我们没办法提取相应的路径。也就是根据当前的图，并不能提取出最短路 $C \to A \to B \to E$，因为只知道每个节点的最小距离，却不知道它们之前的节点是谁。

这个问题也是很久之前就被解决的，我们可以在算法中增加一个小细节，在每个节点更新最短距离时，同时更新它的前一个节点。这样，就可以从任意点开始回溯，得到到达该点的相应的最短路。

例如，我们由图 9.11 就可以通过存储当前节点的前一个节点的信息，得到如图 9.12 所示的图。

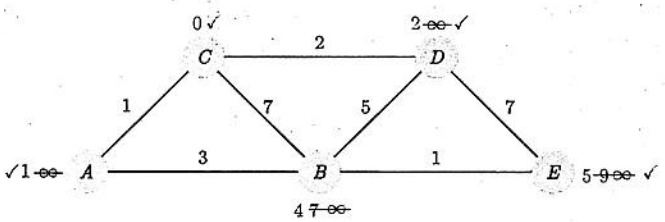


图 9.11　算法迭代完成

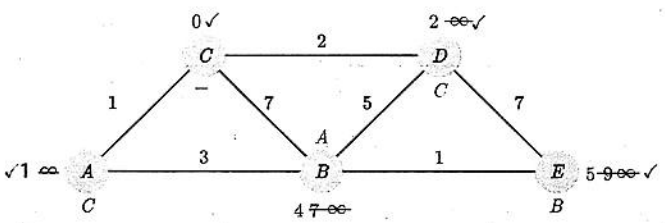


图 9.12　存储前序节点

图 9.12 中，以点 E 为例，向后回溯，点 E 的前一个点是 B，点 B 的前一个点是点 A，点 A 的前一个点是 C，C 点是我们选择的起点。因此得到最短路径 $C \to A \to B \to E$，相应的最短距离为点 E 的标签信息里面存储的最短距离 5。

该算法还有一些其他版本的解释。一般也把上述算法用 T（temporarily）标号和 P（permanent）标号来表示（临时标号和永久标号）。未被探索的节点的最短距离等信息对应 T 标号，已探索的节点的信息对应 P 标号。当图中所有节点的标号都被修改成 P 标号时，算法结束。

9.2　Dijkstra 算法步骤及伪代码

下面给出 Dijkstra 算法的详细描述。

（1）选择一个起始点，将该点作为当前点并将其最短距离标记为 0，将其余所有点的最短距离标记为 ∞，将所有节点的前一个节点标记为 null。

（2）选择所有未被探索的节点中最短距离最小的节点作为新的当前点 C。

（3）对当前节点 C 的所有邻居 $i \in N$ 进行下面的操作：将当前节点 C 的最小距离加上 C 与邻居 i 之间的边的权重。如果两者的和小于当前邻居 i 的最小距离，则修改邻居 i 的最短距离为之前两者的和，并更新该邻居 i 的前一个节点为当前节点 C。

（4）将当前节点 C 标记为已探索。

（5）如果图中还存在未被访问的节点，则转向步骤 2。否则算法结束。

其伪代码如下：

Algorithm 2 Dijkstra Algorithm

```
1:  function Dijkstra Algorithm(Graph, source s)
2:      for 点 v ∈ Graph do
3:          d_v ← ∞
4:          p_v ← undefined
5:      end for
6:      d_s ← 0
7:      Q ← Graph 中的所有点的集合
8:      while Q 非空 do
9:          u ← Q 中 d 最小的点
10:         Q ← Q\{u}
11:         for u 的每个邻居节点 v do
12:             d_temp ← d_u + c_{u,v}
13:             if d_temp < d_v then
14:                 d_v ← d_temp
15:                 p_v ← u
16:             end if
17:         end for
18:     end while
19:     return p
20: end function
```

其中，图中每个节点 i 对应的 d_i 表示点 i 的最短距离；p_i 表示点 i 的前一个节点。$c(u,v)$ 表示点 u,v 之间的距离或者成本。

9.3 Python 实现 Dijkstra 算法

9.3.1 网络数据准备

下面用之前介绍 SPP 的对偶那一章的例子作为测试数据，网络图如图 9.13 所示。

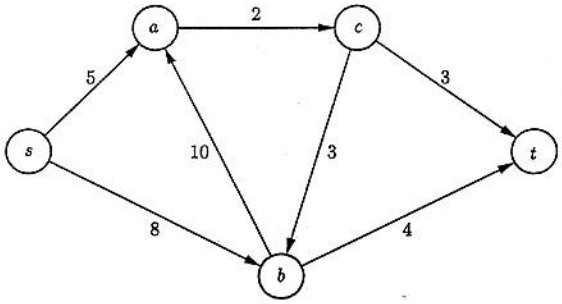


图 9.13 SPP：小算例

实现代码如下：

DataPreparation

```
import pandas as pd
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
import copy
import re
import math

Nodes = ['s', 'a', 'b', 'c', 't']

Arcs = {('s','a'): 5
        ,('s','b'): 8
        ,('a','c'): 2
        ,('b','a'): 10
        ,('c','b'): 3
        ,('b','t'): 4
        ,('c','t'): 3
       }

# 构建有向图对象
Graph = nx.DiGraph()
cnt = 0
pos_location = {}
```

```
for name in Nodes:
    cnt += 1
    X_coor = np.random.randint(1, 10)
    Y_coor = np.random.randint(1, 10)
    Graph.add_node(name
                , ID = cnt
                , node_type = 'normal'
                , demand = 0
                , x_coor = X_coor
                , y_coor = Y_coor
                , min_dis = 0
                , previous_node = None
                )

    pos_location[name] = (X_coor, Y_coor)
# 增加图的边界
for key in Arcs.keys():
    Graph.add_edge(key[0], key[1]
                   , length = Arcs[key]
                   , travelTime = 0
                  )
```

9.3.2 Dijkstra 算法实现

这里的 Dijkstra 算法的代码逻辑基于前面给出的伪代码。

Dijkstra Algorithm

```
def Dijkstra(Graph, org, des):
    # 定义bigM
    bigM = 1000000
    # 将每个点对应的最小距离初始化为无穷大以及初始化队列
    Queue = []
    for node in Graph.nodes:
        Queue.append(node)
        if(node == org):   # 注意，这里的node就直接是node的名字
            Graph.nodes[node]['min_dis'] = 0
        else:
            Graph.nodes[node]['min_dis'] = bigM
    # 循环开始
    while(len(Queue) > 0):
        # 选取下一个节点 : 寻找具有最小 min_dis的节点
        current_node = None
```

```
        min_dis = bigM
        for node in Queue:
            if(Graph.nodes[node]['min_dis'] < min_dis):
                current_node = node
                min_dis = Graph.nodes[node]['min_dis']
        if(current_node != None):
            Queue.remove(current_node)
        # 对每个邻居进行循环
        for child in Graph.successors(current_node):
            # 更新每个节点的 min_dis
            arc_key = (current_node, child)
            dis_temp = Graph.nodes[current_node]['min_dis'] + Graph.edges
          [arc_key]['length']
            if(dis_temp < Graph.nodes[child]['min_dis']):
                Graph.nodes[child]['min_dis'] = dis_temp
                Graph.nodes[child]['previous_node'] = current_node

    opt_dis = Graph.nodes[des]['min_dis']
    current_node = des
    opt_path = [current_node]
    while(current_node != org):
        current_node = Graph.nodes[current_node]['previous_node']
        opt_path.insert(0, current_node)

    return Graph, opt_dis, opt_path
```

9.3.3 算例测试

测试结果如下：

results

```
Graph, opt_dis, opt_path = Dijkstra(Graph, 's', 't')
print('optimal distance : ', opt_dis)
print('optimal path : ', opt_path)

[Out]:
optimal distance : 10
optimal path : ['s', 'a', 'c', 't']
```

测试较大的算例，只需要将代码中的网络数据替换就可以。

9.4 拓展

Dijkstra 算法实际上是一种标号算法（Label setting algorithm），该算法不能求解网络中含有负环的算例，因为 Dijkstra 算法中，标签一旦被设置成永久标签（P 标签）就无法更改。针对网络中有负环的案例，我们可以采用 Bellman-Ford 算法进行求解。Bellman-Ford 算法属于 Label correction algorithm。Label setting algorithm 和 Label correction algorithm 都是基于动态规划思想的算法，且都是精确算法，得到的一定是最优解。这些算法在运筹优化精确算法中有着非常重要的地位，也频繁地出现在一些顶级期刊论文当中。关于它们更多的拓展，这里不做展开，感兴趣的读者可以阅读相关的参考文献（Zhan and Noom, 1998; Cherkassky et al., 1996; Dial et al., 1979）。

第10章　分支定界算法

大家对线性规划都非常熟悉了，但是实际生产活动中，许多问题都是离散的，因此更常见的是整数规划或者混合整数规划。从本章开始，我们就聚焦在混合整数规划的精确求解算法及其实现上。本书介绍的所有精确算法中，分支定界算法是最基本、最底层的算法，其他若干算法都是以分支定界算法为基础的拓展。分支定界算法由伦敦政治经济学院 Ailsa Land 和 Alison Doig 于 1960 年提出（Land and Doig 1960），但是当时她们并没有称其为分支定界算法。而后 John D. C. Little 等在 1963 年发表的关于 TSP 的研究中，首次使用了分支定界方法（branch and bound method）的名称（Little it et al., 1963）。到了今天，分支定界算法已经成为运筹优化领域最著名的算法之一。

10.1　整数规划和混合整数规划

我们使用文献（Winston 和 Goldberg，2004）的第 516 页的例子来开始本章的论述。

某家具公司需要生产一批桌子和椅子。一张桌子需要 1 小时的劳动力和 9 平方英尺的木板，一把椅子需要 1 小时的劳动力和 5 平方英尺的木板。目前，公司可供使用的资源为 6 小时的劳动力和 45 平方英尺的木板。生产每张桌子可以获得 8 美元的利润，生产每把椅子可以获得 5 美元的利润。请将上述问题建模成为整数规划模型，使得该公司的利润达到最大。

要求解该问题，首先引入下面 2 个决策变量：

$$x_1 = \text{生产桌子的数量}$$
$$x_2 = \text{生产椅子的数量}$$

由于桌子和椅子数量必须是整数，因此可以将该模型表示为下面的整数规划：

$$\begin{aligned} \max\ z =& 8x_1 + 5x_2 \\ \text{s.t.}\quad & x_1 + x_2 \leqslant 6 \\ & 9x_1 + 5x_2 \leqslant 45 \\ & x_1, x_2 \geqslant 0,\ x_1, x_2 \text{ 为整数} \end{aligned} \tag{10.1}$$

模型 (10.1) 中，决策变量全部是整数，这种类型的整数规划叫作纯整数规划（Pure Integer Programming）。但是如果将上面模型中的变量 x_2 设置为连续变量，该模型就变成

下面的形式：

$$\begin{aligned} \max\ z =& 8x_1 + 5x_2 \\ \text{s.t.}\quad & x_1 + x_2 \leqslant 6 \\ & 9x_1 + 5x_2 \leqslant 45 \\ & x_1, x_2 \geqslant 0,\ x_1 \text{ integer} \end{aligned} \tag{10.2}$$

模型 (10.2) 中既有连续变量，又有整数变量，我们把它称为混合整数规划（Mixed Integer Programming）。

如果我们将模型 (10.1) 中的变量全部设置成 0-1 变量，那么模型将变成一个 0-1 整数规划。

假如不考虑模型 (10.1) 中的整数约束，那么模型被松弛为

$$\begin{aligned} \max\ z =& 8x_1 + 5x_2 \\ \text{s.t.}\quad & x_1 + x_2 \leqslant 6 \\ & 9x_1 + 5x_2 \leqslant 45 \\ & x_1, x_2 \geqslant 0 \end{aligned} \tag{10.3}$$

模型 (10.3) 是一个线性规划，并且除了变量的取值范围之外，其他约束完全一致，我们将模型 (10.3) 称为模型 (10.1) 的线性松弛（LP relaxation）。

定义 10.1.1 *将整数规划的整数约束和 0-1 约束全部去掉，将得到一个线性规划。该线性规划称为原来整数规划的线性松弛。*

由于 IP 的线性松弛约束更少，因此 LP relaxation 的可行域一定包含 IP 的可行域。对于 max 问题，一定有

$$z^*_{\text{IP}} \leqslant z^*_{\text{LP relaxation}} \tag{10.4}$$

整数规划和混合整数规划的可行域都包含一系列离散的点，是极度非凸的。整数规划和混合整数规划都已经被证明是 NP-Complete 问题（虽然在某些特定问题背景下也许不是 NP-Complete）。随着问题规模不断增大，求解难度呈指数级或更剧烈地上升。处理 MIP 的一个最基本、最经典的算法就是分支定界算法。本章我们首先以一个简单的例子来梳理分支定界算法的基本流程，然后给出分支定界算法的完整流程和详细伪代码。最后以分支定界算法求解 VRP 为例，介绍 Python 调用 Gurobi 如何实现分支定界算法。

10.2 分支定界算法求解混合整数规划

分支定界算法是一种通过将可行域分成较小的子区域的方法[①]。在具体介绍分支定界算法之前，我们首先给出一个非常重要但是基础的发现。如果求解一个 IP 的线性松弛，得到了一个变量全是整数的解，那这个线性松弛的最优解也同时是 IP 的最优解。

① 本章部分参考 http://web.tecnico.ulisboa.pt/mcasquilho/compute/_linpro/TaylorB_module_c.pdf.

接下来简单描述一下分支定界算法的大体思想。假设我们考虑一个 max 问题的 IP，该 IP 的线性松弛称为 LPr。我们不直接求解 IP，而是先求解 LPr，如果 LPr 的解不满足 IP 的整数约束，则 LPr 的目标函数 z_{LPr} 一定是 IP 的最优目标值的一个上界，记为 z^u（因为 LPr 少了整数约束，其可行域一定包含了 IP 的所有可行解，因此 LPr 的最优解一定大于或等于 IP 的最优解）；而 IP 的任意一个可行解（整数解）的目标函数必然小于或等于其最优解的目标函数，因此是 IP 的最优目标函数的一个下界，记作 z^l。假设 IP 的最优目标函数值为 z^*，则必有 $z^l \leqslant z^* \leqslant z^u$。分支定界法就是将 LPr 的可行域经过不断地分支，划分成小的子区域，并求解每个子区域对应的问题，利用它们的目标函数信息，不断增大 z^l、减小 z^u，最终使其相遇（或者足够靠近），从而得到 IP 的最优解 z^*。总体上来讲，该方法就是一种分而治之的方法（divide and conquer）。

我们考虑下面的 IP （Russell and Taylor-Iii，2008）：

$$\max\ z = 100x_1 + 150x_2 \tag{10.5}$$

$$\text{s.t.}\quad 2x_1 + x_2 \leqslant 10 \tag{10.6}$$

$$3x_1 + 6x_2 \leqslant 40 \tag{10.7}$$

$$x_1, x_2 \geqslant 0 \tag{10.8}$$

$$x_1, x_2 \text{ integer} \tag{10.9}$$

首先将上述模型的整数约束 (10.9) 松弛掉，变成该问题的线性松弛。我们将该问题称为子问题 1 （Subproblem 1），如下：

$$\max\ z = 100x_1 + 150x_2 \tag{10.10}$$

$$\text{s.t.}\quad 2x_1 + x_2 \leqslant 10 \tag{10.11}$$

$$3x_1 + 6x_2 \leqslant 40 \tag{10.12}$$

$$x_1, x_2 \geqslant 0 \tag{10.13}$$

尝试求解子问题 1，如果最优解是整数，则该最优解也同时是 IP 的最优解，但是很遗憾，子问题 1 的最优解为

$$x_1 = 2.22,\ x_2 = 5.56,\ Z = 1055.56$$

将其圆整，发现也是一个可行解，如下：

$$x_1 = 2,\ x_2 = 5,\ Z = 950$$

分支定界方法就是使用节点（Nodes）和分支（Branches）的树形图（Tree）来分割可行域的，利用这些节点和分支来控制求解的过程。我们把这个树形图叫作分支定界树（Branch and Bound tree，BB tree 或者 B&B tree 或者 BnB tree）。这里把 BB tree 中的每个节点叫作一个节点（Node），每次划分可行域的操作叫作分支（Branching）。

BB tree 的第一个节点对应的上界（Upper Bound，UB）和下界（Lower Bound，LB）就是 IP 的线性松弛的小数解和经过圆整的整数解，因此有 $z^l = 950, z^u = 1055.56$。IP 的最优解一定在 z^l 和 z^u 之间。本节统一用 UB 和 LB 分别表示上界和下界，如图 10.1 所示。图 10.1 中的 UB 和 LB 指的是每个节点的 UB 和 LB，并不是全局的 UB 和 LB。当然，根节点的 UB 和 LB 同时也是全局的 UB 和 LB，但其他节点的就不是了。

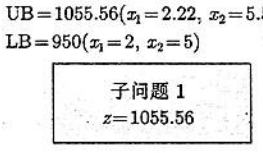


图 10.1 BB tree 1：初始节点（根节点）

IP 的线性松弛解给出了 IP 的一个上界，IP 的最优解一定不可能比这个值更大。而我们通过向下取整获得的解，给出了 IP 的解的一个下界，我们忽略比这个下界还小的整数解，因为比这个下界还小的整数解，一定不是最优解。下面进行分支定界算法的操作。

分支定界算法的第一步是从当前松弛解中创建两个当前节点线性松弛问题的可行域的子集。这是通过考查每个决策变量的松弛解的取值来完成的，我们来看看哪一个变量距离舍入后的整数值最远（即哪个变量的分数部分最大）。在本例的根节点中，比较 x_1 和 x_2 的取值，发现 5.56 的 0.56 部分是最大的分数部分；因此，x_2 将是我们要“分支”的变量。

由于 x_2 必须是一个整数，因此我们可以创建如下 2 个互斥的约束：

$$x_2 \leqslant 5$$

$$x_2 \geqslant 6$$

换句话说，x_2 可以是 0、1、2、3、4、5 或 6、7、8 等，但不能是 5 到 6 之间的值，例如 5.56。这 2 个新的约束表示我们的问题的可行域的 2 个子集。这些约束中的每一个都将添加到相应的子问题（子节点）对应的线性规划模型中。这一个分支，只是割掉了 $5 < x_2 < 6$ 的部分，也就是割去了非可行解的部分，并没有割去任何整数可行解。因此，该分支操作不影响最优性。

分支的顺序如图 10.2 所示。我们为根节点 1 创建 2 个子节点 2 和 3。

首先节点 2（或者子问题 2）是通过求解添加了约束 $x_2 \leqslant 5$ 的下述线性规划问题：

$$\max \ z = 100x_1 + 150x_2 \tag{10.14}$$

$$\text{s.t.} \quad 2x_1 + x_2 \leqslant 10 \tag{10.15}$$

$$3x_1 + 6x_2 \leqslant 40 \tag{10.16}$$

$$\boxed{x_2 \leqslant 5} \tag{10.17}$$

$$x_1, x_2 \geqslant 0 \tag{10.18}$$

上述松弛了整数约束的线性规划的最优解为 $x_1 = 2.5,\ x_2 = 5,\ z = 1000$。

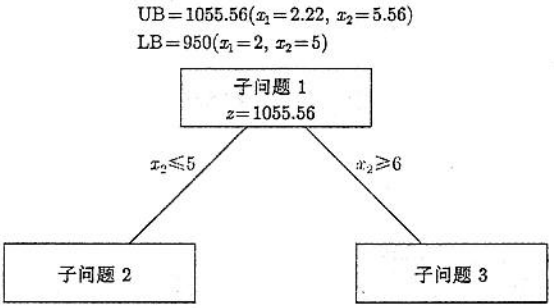


图 10.2 BB tree 2：分支 x_2

类似地，节点 3 的解是将子问题 1 添加分支约束 $x_2 \geqslant 6$，得到下面的松弛了整数约束的线性规划：

$$\max \ z = 100x_1 + 150x_2 \tag{10.19}$$

$$\text{s.t.} \quad 2x_1 + x_2 \leqslant 10 \tag{10.20}$$

$$3x_1 + 6x_2 \leqslant 40 \tag{10.21}$$

$$\boxed{x_2 \geqslant 6} \tag{10.22}$$

$$x_1, x_2 \geqslant 0 \tag{10.23}$$

其最优解为 $x_1 = 1.33,\ x_2 = 6,\ z = 1033.33$。

可以得知，上面的分支操作，将子问题 1 的可行域，分割成了 2 个子区域。然后通过添加分支约束重新求解子问题 2 和 3，分别得到了 2 个子区域的最优解。

注意图 10.2 中的子问题 2 节点，最优解为 $x_1 = 2.5, x_2 = 5, z = 1000$，因此该节点的上界为 1000。在子问题 3 对应的节点上，$x_1 = 1.33, x_2 = 6, z = 1033.33$，因此节点 3 的上界为 1033。由于这些松弛问题的解都不是整数，因此下界依然是 950 不变，还是在节点 1 获得的下界，如图 10.3 所示。

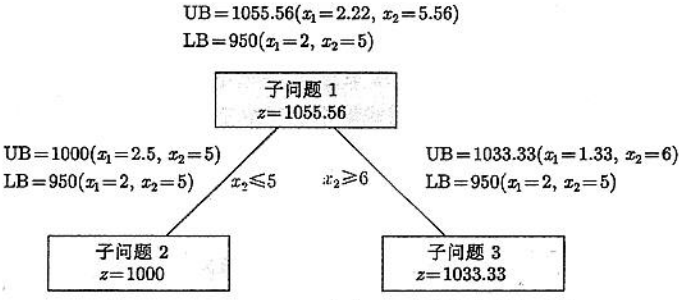


图 10.3 BB tree 2：分支 x_2 及子节点的上下界

由于还没有找到最优可行整数解，因此必须继续从节点 2 或节点 3 分支。从图 10.3可以发现，如果从节点 2 分支，最高可能达到的目标值是 1000（UB）。但是，如果从节点 3

分支，则可能会有更高的最大值 1033。因此，可以从节点 3 分支。通常，人们总是从具有最大上界的节点开始分支。

这也是分支定界算法中的一个重要步骤，在前一次分支之后，首先求解新产生的 2 个子节点的线性松弛问题，然后通过比较更新每个节点的 UB 和 LB。更新完之后，再次通过比较各个节点的 UB 选出下一步需要分支的节点。

现在，在节点 3 上重复先前在节点 1 上执行的分支步骤。首先，选择具有最大小数部分值的变量。由于 x_2 是整数，因此只能选择 x_1，从 x_1 发展出 2 个新的约束

$$x_1 \leqslant 1$$

$$x_1 \geqslant 2$$

该步骤将会生成新的 BB tree，如图 10.4 所示。

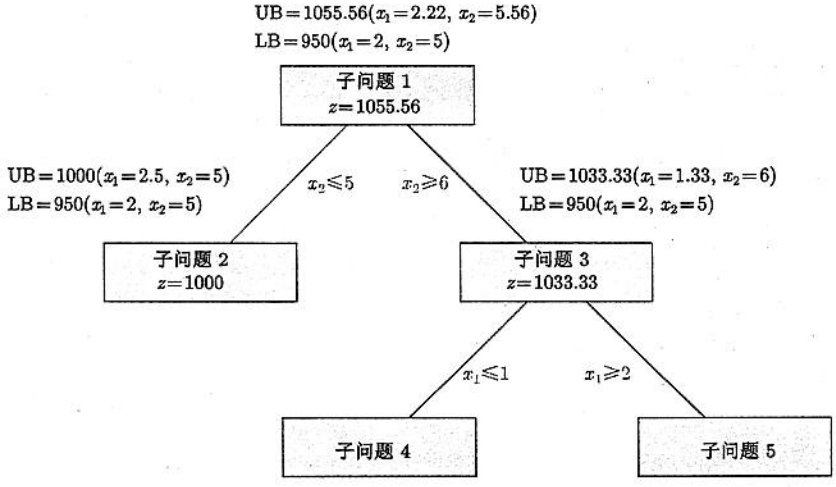


图 10.4 BB tree 2：分支 x_1

接下来需要在节点 4 和 5 处求解添加了新约束的线性松弛问题。（但是，不要忘记该模型不是原始模型，而是先前添加了约束 $x_2 \geqslant 6$ 的原始模型）。首先是节点 4 的模型：

$$\max \ z = 100x_1 + 150x_2 \tag{10.24}$$

$$\text{s.t.} \quad 2x_1 + x_2 \leqslant 10 \tag{10.25}$$

$$3x_1 + 6x_2 \leqslant 40 \tag{10.26}$$

$$\boxed{x_2 \geqslant 6} \tag{10.27}$$

$$\boxed{x_1 \leqslant 1} \tag{10.28}$$

$$x_1, x_2 \geqslant 0 \tag{10.29}$$

该问题的最优解是 $x_1 = 1, x_2 = 6.17, z = 1025.5$。

接下来是节点 5 的模型：

$$\max\ z = 100x_1 + 150x_2 \tag{10.30}$$

$$\text{s.t.}\quad 2x_1 + x_2 \leqslant 10 \tag{10.31}$$

$$3x_1 + 6x_2 \leqslant 40 \tag{10.32}$$

$$\boxed{x_2 \geqslant 6} \tag{10.33}$$

$$\boxed{x_1 \geqslant 2} \tag{10.34}$$

$$x_1, x_2 \geqslant 0 \tag{10.35}$$

该问题无解。可以得知，节点 5 已经不可能再往下分支了，因此称节点 5 已被查明 (或洞悉)，英文术语为 fathomed。当对一个节点对应的子问题的进一步分支不能产生任何有用信息时，就说这个节点已经被查明（或洞悉）了。为了表示这个事实，在该节点旁边用 ✖ 表示。并且，将其剪枝（prune）。

因此 BB tree 变化为如图 10.5 所示的形式。

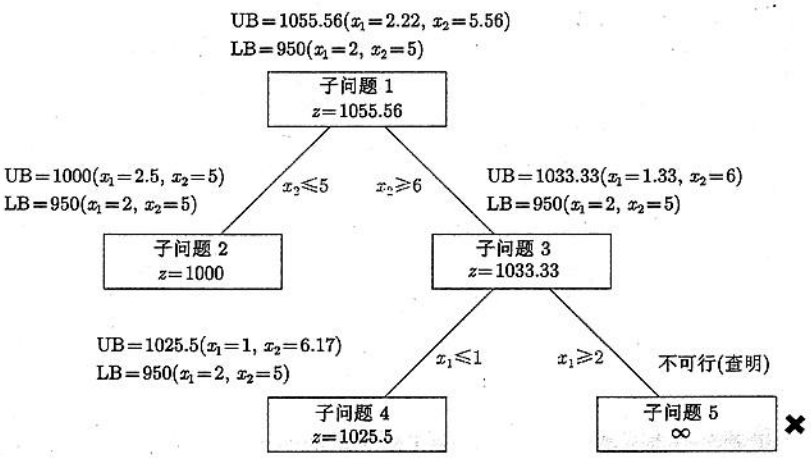


图 10.5　BB tree 3：分支 x_1 及子节点的上下界

图 10.5 中的 BB tree 表明仍未达到最佳整数解；因此，必须重复之前的分支步骤。由于节点 5 不可行，因此在节点 4 和 5 的上限之间没有比较。比较节点 4 和 2，由于节点 4 的 UB 更大，表示节点 4 更有可能获得更大的整数解，因此选择从节点 4 分支。接下来，由于 x_1 是整数值，因此默认选择 x_2。分支 x_2 产生的 2 个新约束是

$$x_2 \leqslant 6$$

$$x_2 \geqslant 7$$

这一步分支产生的新的 BB tree 如图 10.6 所示。

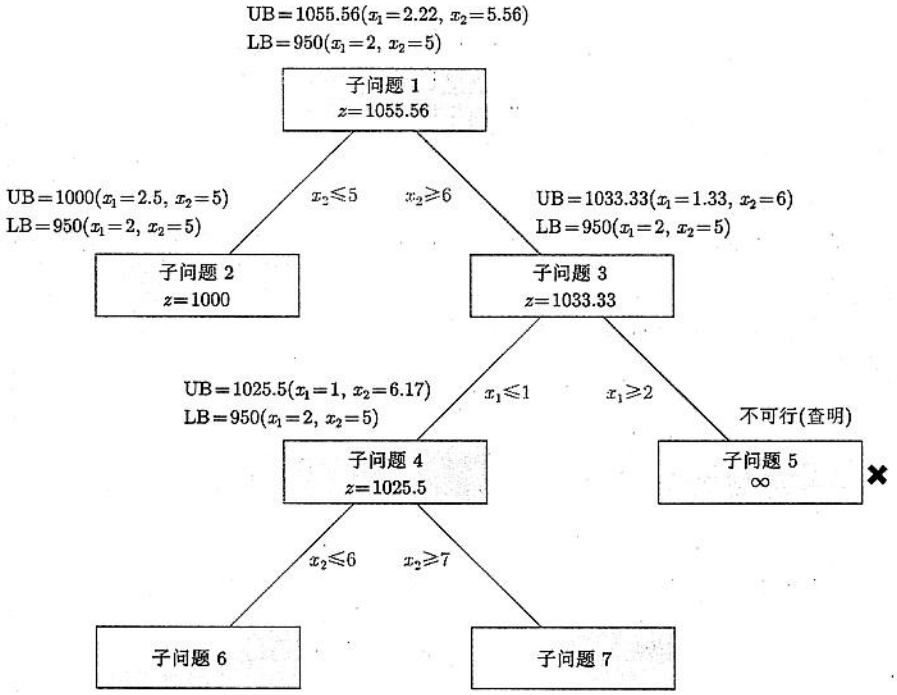


图 10.6 BB tree 4：分支 x_2

接下来需要在节点 6 和 7 处求解添加了新约束的线性松弛问题。（同样地，需要注意该模型不是原始模型，而是先前添加了约束 $x_2 \geqslant 6$ 和约束 $x_1 \leqslant 1$ 的原始模型。）首先是节点 6 的模型：

$$\max z = 100x_1 + 150x_2 \tag{10.36}$$

$$\text{s.t. } 2x_1 + x_2 \leqslant 10 \tag{10.37}$$

$$3x_1 + 6x_2 \leqslant 40 \tag{10.38}$$

$$\boxed{x_2 \geqslant 6} \tag{10.39}$$

$$\boxed{x_1 \leqslant 1} \tag{10.40}$$

$$\boxed{x_2 \leqslant 6} \tag{10.41}$$

$$x_1, x_2 \geqslant 0 \tag{10.42}$$

该分支的最优解为 $x_1 = 1, x_2 = 6, z = 1000$。

接下来是节点 7 的模型：

$$\max z = 100x_1 + 150x_2 \tag{10.43}$$

$$\text{s.t. } 2x_1 + x_2 \leqslant 10 \tag{10.44}$$

$$3x_1 + 6x_2 \leqslant 40 \tag{10.45}$$

$$\boxed{x_2 \geqslant 6} \tag{10.46}$$

$$\boxed{x_1 \leqslant 1} \tag{10.47}$$

$$\boxed{x_2 \geqslant 7} \tag{10.48}$$

$$x_1, x_2 \geqslant 0 \tag{10.49}$$

该问题无解。因此节点 7 也已被查明（fathomed），同样在该节点旁边用 ✖ 表示。相应地，也将其剪枝（prune）。

因此更新 BB tree，如图 10.7 所示。

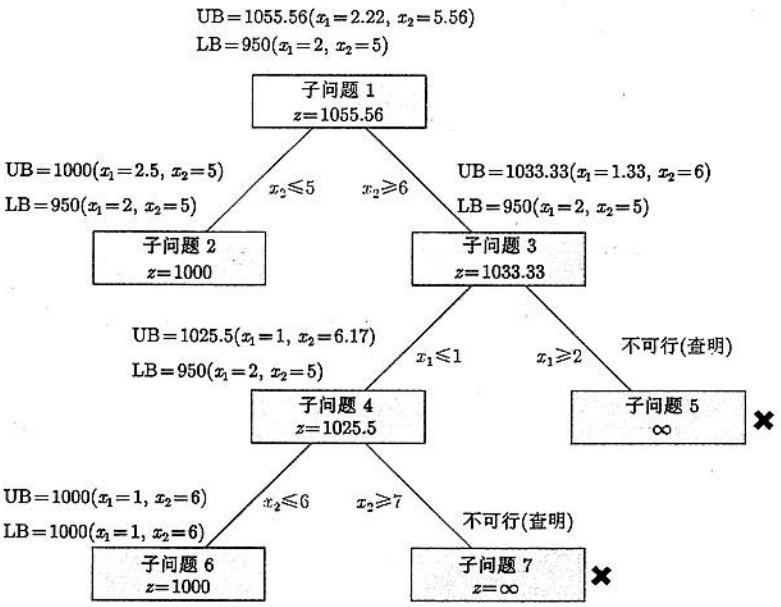


图 10.7　BB tree 4：分支 x_2 及子节点的上下界

该步的左分支节点 6 得到了一个整数解 $x_1 = 1, x_2 = 6, z = 1000$，该解也是该点的线性松弛模型的最优解。因此，$z = 1000$ 也表示节点 6 可以取得的上界或者整数解目标值的上界。而该解也是一个整数解，整数解一定是 IP 的一个 LB，并且该值大于当前的 LB（950），因此需要更新该点的下界为 LB = 1000，这也是目前为止 BB tree 得到的最佳整数解 $x_1 = 1, x_2 = 6, z = 1000$。此时 LB = UB = 1000，并且其他所有叶子节点 2, 5, 7 显示不可能通过分支得到更好的解。节点 5 和 7 不可行，将其剪枝（Pruned by infeasibility）。节点 2 处的 UB = 1000，所以通过分支不可能获得更好的整数解，也将其剪枝（Pruned by bound）。通过剪枝过程，我们在节点 6 处得到最优整数解 $x_1 = 1, x_2 = 6, z = 1000$。当然，得到最优整数解后，节点 6 也被剪枝（Pruned by optimality）。所有叶子节点都被剪枝，算法终止。

一般来讲，当一个节点得到了一个整数解，并且该节点的 UB 大于或者等于其他任何叶子节点（即分支结束的节点）的 UB 时，该点的整数解就是全局最优整数解。

这里需要强调一下查明（洞悉，fathomed）这个概念。查明（洞悉）的节点需要经过查明条件测试，也就是如果节点满足以下任意一个条件，则此节点可查明而不再被考虑。之前介绍时，当对一个节点对应的子问题的进一步分支不能产生任何有用信息时，我们就说这个节点已经被查明了。但是这个定义很模糊。现在我们给出一个判断一个节点已查明的判断条件。对于下面 3 个条件（Winston and Goldberg，2004）：

（1）该节点不可行；

（2）此节点得到了一个整数解；

（3）该节点的线性松弛问题的最优值 z 不超过当前的全局 LB。

满足 3 个条件中的任意一个，则可断定该节点是已查明的。查明的节点无须再进行分支，我们就可以将其剪枝（prune）。

根据分支定界算法的过程，可以有一些发现，在整个迭代的过程中，UB 下降，LB 上升，直到 UB = LB 或者 UB − LB $< \epsilon$（ϵ 为很小的数）时，算法停止。此外，还有下面的一些发现。

（1）当前节点的最优解是一个局部最优解。

（2）全局最紧的 LB 等于所有剩余叶子节点的局部最优解的最大值（因为 max 问题的 LB 都是整数可行解，因此是最大值）。

（3）当前节点的线性松弛解是一个局部 UB。

（4）全局最紧的 UB 是所有剩余的节点的局部 UB 的最大值（因为每一个子问题中都有可能包含整数可行解，因此就有得到更大目标函数值的可能性）。

总结上面的观察，就会得到下面的定理。

定理 10.2.1 （Wolsey，1998）假设一个整数规划 $z = \max\{cx, x \in S\}$ 的可行域 S 被分成 K 个小的子集，即 $S = S_1 \cup S_2 \cup \cdots \cup S_k$，并且令 $z^k = \max\{cx, x \in S_k\}, \forall k \in K$，$\bar{z}^k$ 是 z^k 的一个上界，$\underline{z}^k$ 是 z^k 的一个下界，则 $\bar{z} = \max_k \bar{z}^k$ 是 z 的一个上界，且 $\underline{z} = \max_k \underline{z}^k$ 是 z 的一个下界。

定理 10.2.1 中 S 可以是根节点，也可以是 BB tree 中某几个子节点的父节点。

结合上面的定理和查明的定义，能够帮助我们做剪枝（prune）。这里给出下面 3 个剪枝的情况（max 问题）。

（1）**根据最优性剪枝**（Pruned by optimality）：对于子问题 k，$z^k = \max\{cx, x \in S_k\}$ 得到了整数可行解；此时该子问题同时也是被查明的。

（2）**根据 bound 剪枝**（Pruned by bound）：如果子问题 k 的上界不大于全局下界，即 $\bar{z}^k \leqslant \underline{z}$；这也表明该节点继续分支不会产生比当前 LB 更好的解，因此该节点也是属于被查明的节点。

（3）**根据是否可行剪枝**（Pruned by feasibility/infeasibility）：如果子问题 k 是非可行的，可以将其剪枝；当然，非可行节点同样属于被查明的节点。

下面给出 3 个具体例子，让大家更直观地理解剪枝。分别如图 10.8 ～ 图 10.10 所示。

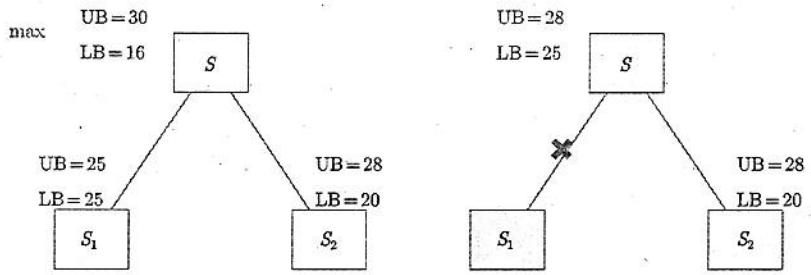


图 10.8　剪枝：根据最优性剪枝 (Pruned by optimality)

在图 10.8中，将父节点 S 分割成了 2 个子集，并且给出了每个节点的 UB 和 LB。根据上述定理，父节点的 UB 为 $\bar{z} = \max_k \bar{z}^k = \max_k\{25, 28\} = 28$, LB 为 $\bar{z} = \max_k \underline{z}^k = \max_k\{25, 20\} = 25$，另外注意到 S_1 的 UB 和 LB 相等，因此该节点已被查明，因此可以根据最优性将其剪枝。

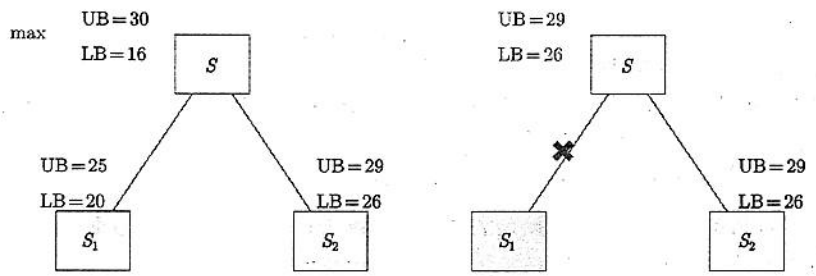


图 10.9　剪枝：根据界限剪枝（Pruned by bound）

在图 10.9 中，仍然将父节点 S 分割成了 2 个子集，同样给出了每个节点的 UB 和 LB。因此，父节点的 UB 为 $\bar{z} = \max_k \bar{z}^k = \max_k\{25, 29\} = 29$, LB 为 $\underline{z} = \max_k \underline{z}^k = \max_k\{20, 26\} = 26$，注意到 S 的 LB（最优解的 LB）为 26，而点 S_1 的 UB 仅为 25，也就是说 S_1 中不可能有最优解，该节点已被查明，因此可以根据 bound 将其剪枝。

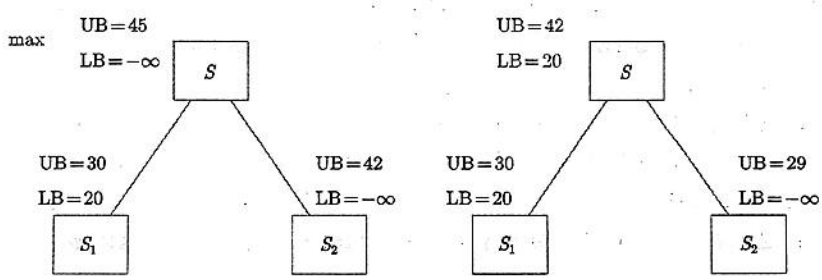


图 10.10　不能剪枝的情形

在图 10.10 中，将父节点 S 分割成了 2 个子集，也给出了每个节点的 UB 和 LB。同理，父

节点的 UB 为 $\bar{z}=\max_k \bar{z}^k=\max_k\{30,42\}=42$, LB 为 $\bar{z}=\max_k \underline{z}^k=\max_k\{20,-\infty\}=20$。根据现在的信息，无法得到其他结论，因此需要继续对 S_1 和 S_2 进行分支探索。

另外，在上述几个例子中，在选择分支节点的时候，是比较随意的，直接选择了剩余的并且未被查明的节点中 UB 最大的节点作为分支节点。但是这个方法并不一定是最好的，还有很多其他的选择方法。在变量数量比较多时，不同的选择下一个分支节点的方法有时候会导致运算时间有巨大的差异。具体更加全面的总结在后面的内容再予以介绍。

1. max 问题中的上界和下界

在 max 问题中，原问题的任意可行解是最优解的一个下界（LB），假设可行解的目标函数为 y，则最优值 y^* 一定有 $y^*\geqslant y$。

迄今为止获得的最好可行解就是最紧的 LB，英文文献中叫作 incumbent。

只考虑一部分约束 $X'\subset X$ 的解，是原问题的一个上界（UB）。如果 UB 小于或等于目前得到的最好的 LB，就没有必要继续分支，也说明得到了最优解，这个 UB 也就是最优 UB。

2. min 问题中的上界和下界

在 min 问题中，原问题的任意可行解是最优解的一个上界（UB），假设可行解的目标函数为 y，则最优值 y^* 一定有 $y^*\leqslant y$。

迄今为止获得的最好解就是最紧的 UB，同样，称其为 incumbent。

只考虑一部分约束 $X'\subset X$ 的解，是原问题的一个下界（LB）。如果 LB 大于或等于目前得到的最好的 UB，就没有必要继续分支，也说明得到了最优解，这个 LB 也就是最优 LB。

10.3 分支定界算法的一般步骤和伪代码

用分支定界算法求解 max 的 IP 问题的一般步骤如下（$\leqslant$ 约束）。

（1）求得原问题线性松弛模型的最优解，作为节点 1；

（2）在节点 1 处，将线性松弛的最优解作为 UB，将线性松弛的解向下圆整获得的解作为 LB；

（3）选择分数部分最大的小数变量进行分支，构建两个将可行域按照变量的整数限制分割成两个子区域的分支约束。也就是构造一个 $\leqslant$ 约束和一个 $\geqslant$ 约束；

（4）在 BB tree 里创建两个新的节点，一个是对应 $\geqslant$ 约束的节点，另一个是对应 $\leqslant$ 约束的节点；

（5）求解这两个新的节点处的线性松弛模型；

（6）这个松弛的解是每个点的 UB，并且已经得到的整数解（在任意节点上）是 LB；并且根据查明的条件，判断该节点是否已被查明；

（7）当一个节点得到了一个整数解，并且该节点的 LB 大于或者等于其他任何叶子节点的 UB 时（也就是大于或等于全局 UB 时），我们就求得了全局最优整数解；如果该节

点的解不是整数解，我们就从所有节点中 UB 最大的节点处开始分支；

（8）回到步骤（3）。

对于最小化模型，可以在根节点（root node）将线性松弛的解四舍五入得到初始整数可行解，并将上下界颠倒即可。

分支定界算法也有另外版本的解释，可供参考（参考自 https://baike.baidu.com/item）。

（1）如果问题的目标为最小化，则设定最优解的初始值 $z=\infty$；

（2）根据分支法则（Branching rule），从尚未被查明节点（局部解）中选择一个节点，并在此节点创建几个新的子节点；

（3）计算每一个新分支出来的子节点的下界；

（4）对每一节点进行洞悉条件测试，若节点满足以下任意一个条件，则此节点可查明而不再被考虑：

① 此节点的下界值大于或等于 z 值；

② 在此节点已找到该节点的最优可行整数解（如果该条件成立，则需比较此可行解与 z 值，若前者较小，则需更新 z 值为该可行解的目标值）；

③ 此节点不可行；

（5）判断是否仍有尚未被洞悉的节点，如果有，则进行步骤二；如果已无尚未被查明的节点，则算法停止，得到最优解。

分支定界算法是针对组合优化问题的非常有效的求解方法。

还有一些文献中将分支定界过程中未被查明的节点称为 active node，这些 active node 就是等待被探索的节点。分支（Branching）就是指探索一个 active node 节点的子树（subtree），而定界（Bounding）则是指评估 active node 的子树的解的界限。

下面给出一个非常基础版本的分支定界算法的伪代码。

Algorithm 3 分支定界算法 1

```
rootNode ← 求解 IP 的线性松弛
activeNodeSet ← ∅
if rootNode 的解是可行整数解 then
    bestValue ← rootNode 的目标值
    currentBest ← rootNode 的解
    算法停止, return bestValue, currentBest
else
    activeNodeSet ← { rootNode }
end if
bestValue ← Null
currentBest ← Null
while activeNodeSet 非空 do
```

```
    选择一个分支节点, 节点 k ∈ activeNodeSet
    将分支节点 k 从 activeNodeSet 中删去
    生成分支节点 k 的子节点 i, i= 1,2, ··· , n_k(n_k ⩾ 2) 及其各自的最优界限 z_i
    for i = 1, ··· , n_k do
        if z_i 劣于当前最好界限或为空 then
            节点 i 被剪枝 (根据界限剪枝或根据不可行性剪枝)
        else if 子节点的最优解是整数可行解且优于当前最好解 then
            bestValue ← z_i
            currentBest ← 子节点 i
        else
            将子节点 i 添加到 activeNodeSet 中
        end if
    end for
end while
return bestValue, currentBest
```

在上述代码中，currentBest 存储了到目前为止最好的整数解，也就是之前介绍的 incumbent。currentBest 这个解对应的目标函数值就是 bestValue。这个值用于辅助查看每个节点是否有继续分支的必要。节点的上界和下界用来评估一个节点有多好。如果该节点的界限比 currentBest 差，则无须评估该节点的子节点（因此我们不会将其添加到 activeSet）。

或者可以借鉴课本（Wolsey, 1998）的算法流程图，整理出分支定界算法的伪代码（max 问题）。算法流程图如图 10.11 所示。

我们整理出的伪代码如下（max 问题）。

Algorithm 4 分支定界算法 2

```
初始化: 初始问题 S 及其模型 P，将其加入列表 Q
z ← −∞
incumbent x* ← Null
while Q 非空 do
    选择问题 S^i ∈ Q，其模型为 P^i
    求解 P^i 的 LP 松弛
    对偶边界 z^i ← LP 的目标函数值
    x^i (LP) ← LP 的解
    if P^i 空 then
        根据不可行性剪枝
    else if z^i ⩽ z then
        根据界限剪枝
```

else if x^i(LP) 是整数解 **then**
    更新原界限 $\underline{z} = \bar{z}^i$
    更新 incumbent $x^* = x^i$(LP)
    根据最优性剪枝
else
    返回两个子问题 S_1^i 和 S_1^i，其子问题为 P_1^i 和 P_1^i
    $Q \leftarrow Q \cup \{S_1^i, S_1^2\}$
end if
end while
return incumbent x^* optimal

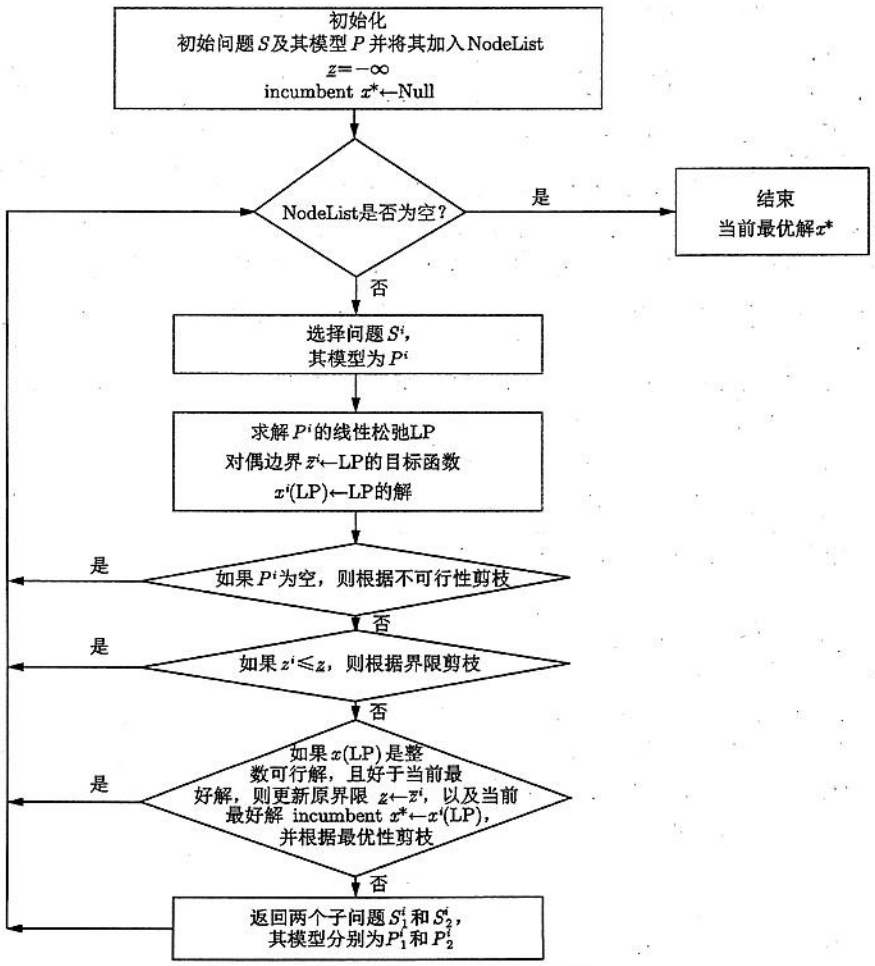


图 10.11　分支定界算法流程图

混合整数线性规划问题也可以使用分支定界算法来解决。只需要将上文中介绍的针对纯整数规划的算法做一些小的改变，就可以用来求解混合整数规划问题。也就是说，我们在求解混合整数规划时，只需对整数约束的变量进行四舍五入以达到初始下界，并且仅对整数变量进行分支即可。

对于 0-1 整数规划，只需要在节点处针对要分支的变量，添加

$$x = 0$$

$$x = 1$$

这样的约束即可。在实际的代码实现中，可以通过设置决策变量的 UB 和 LB 快速完成。比如在 Java 调用 CPLEX 自己动手实现 Branch and Bound 时，就可以调用 `setLB(double lb)` 和 `setUB(double ub)` 来完成分支定界的过程。而 Gurobi 也有同样功能的函数，即 `var.lb = 0.0` 或者 `var.ub = 1.0`，其中 `var` 是决策变量对象。

10.4 分支定界算法执行过程的直观展示

本节以一些图例来直观展示 Branch and Bound 算法的执行过程（以 min 问题为例）。在算法刚开始时，设置好初始的上界 UB 和下界 LB，此时上下界之间的 Gap 非常大，如图 10.12 所示。

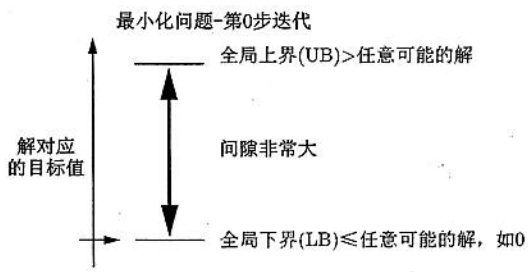


图 10.12 分支定界算法中的界限更新 1

紧接着，在算法执行过程中，我们找到了一些可行解，UB 逐渐缩小，如图 10.13 所示。我们找到的任意一个整数解都可以作为上界。

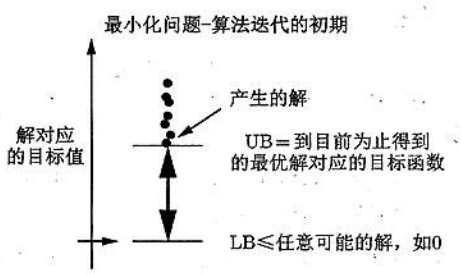


图 10.13 分支定界算法中的界限更新 2

同时，我们可以通过其他手段获得原问题的下界，加快算法收敛。

LB 可以通过线性松弛或者其他方法获得。另外，LB 必须对某些候选集可能实现的最优解进行低估或精确估计。比如可以简单地用叶子节点的线性松弛解作为 LB，或者说，可以通过拉格朗日松弛（后续章节会介绍）来获得一些更紧、质量更好的 LB。随着 LB 的不断更新，UB 和 LB 不断靠近，如图 10.14 所示。

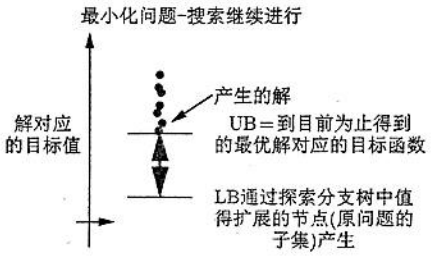


图 10.14　分支定界算法中的界限更新 3

当问题的所有未探索子集的 LB ⩾ UB（到目前为止找到的最好解）时，算法就可以终止了。此时就得到了最优解，如图 10.15 所示。

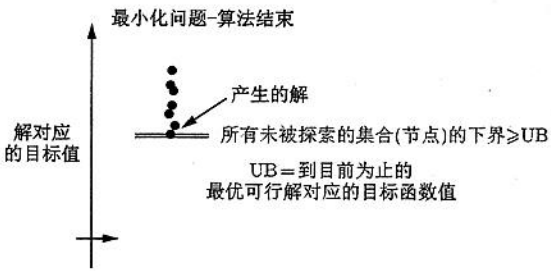


图 10.15　分支定界算法中的界限更新 4

10.5　分支定界算法的分支策略

之前例子中执行第一次分支时，由于 $x_1 = 2.22, x_2 = 5.56$，于是我们选分数部分较大的 x_2 作为分支变量进行分支。但是，是不是一定要选 x_2 呢？以及选 x_2 效果一定更好吗？答案是不一定。分支时，根据不同的规则选择分支变量，会影响求解效率。下面给出几种常见的可选分支策略。

（1）最不可行分支（Most Infeasible Branching）：选取变量取值的分数部分最接近 0.5 的变量作为下一个分支变量。

（2）最大分数值分支（Max Fractional Value Branching）：取最大分数值对应的变量作为分支变量。该方法甚至比随机选取还差。

（3）伪检验数分支（Pseudo Reduced Cost Branching）：通过用启发式的方法近似评估约束的对偶价格，从而近似评估变量的检验数（因此叫伪检验数），最后根据伪检验数来

评估变量分支对目标的影响，以影响大小为基准来选择分支变量。

（4）强分支（Strong branching）：在分支之前首先估计哪个分支会给目标函数带来最大的改进，然后选该变量进行分支。

（5）伪成本分支（Pseudo cost branching）：该策略的基本思想是，在分支的过程中记录每个被选择的分支变量 x_i 分支前后的目标函数的变化，在当前时刻选择下一个分支变量时，根据每个取值为小数的变量的以往的分支相关信息，选择预期会给目标函数带来最大单位改善的变量作为下一个分支变量。

注意：伪成本分支在最初搜索中是没有任何可用信息的，因为在初期几乎没有分支变量。

基于模型的决策变量本身的分支称为 LP based branch and bound。除此之外，还可以针对问题特性或针对其他综合指标进行分支。

10.6 分支定界算法的搜索策略

在 Gurobi 中，可以通过修改混合整数规划模型的参数 `VarBranch` 的值来改变分支策略，语法为 `m.setParam('VarBranch',0)`。`VarBranch` 的取值和含义如下。

- 0，表示伪检验数分支（Pseudo Reduced Cost Branching）；
- 1，表示伪影子价格分支（Pseudo Shadow Price Branching）；
- 2，表示最大不可行分支（Maximum Infeasibility Branching）；
- 3，表示强分支（Strong Branching）。

除此之外，还可以对单个变量的分支优先级进行修改。具体来讲就是修改变量的属性 `BranchPriority`。该属性是一个 int 类型的值，默认情况下，取值越大的变量，分支优先级越高。

在 BB tree 搜索的过程中，我们之前介绍到，选择 UB 最大的未被探明的叶子节点为下一个分支节点。但是我们观察到，这样做并不总是最好的。也有一些其他的遍历策略。常见的 BB tree 策略如下[①]。

（1）深度优先搜索（Depth first search）：该策略能够更快地找到可行解。

（2）广度优先搜索（Breadth first search）：该策略可以更快地剪掉更多的分支。

（3）最好界限优先搜索（Best first search）：LB 或者 UB 最好的节点首先被探索。

（4）其他的启发式规则。

10.7 分支定界算法的剪枝策略

剪枝策略包括如下几个。

（1）查明一个节点（Fathoming a Node）：跟之前的描述一样，满足 3 个查明条件之一的节点，可以被判定为查明，可以用来剪枝。

① 参见 https://www.sciencedirect.com/topics/computer-science/branch-and-bound-algorithm-design，这里介绍了分支定界搜索策略的一些相关内容，供读者参考。

（2）优超关系（Dominance Relations）：如果可以在任何时候确定节点 y 的最佳子节点至少与节点 x 的最佳后代一样好，那么就说 y 优超了 x。

关于分支定界算法的最新进展，参见文献（Morrison it et al., 2016）。

10.8 Python 调用 Gurobi 实现分支定界算法的简单案例

为方便读者理解，我们首先以前文中的简单整数规划为例来实现分支定界算法，然后再以 TSP 和 VRPTW 为例，介绍分支定界算法求解复杂（混合）整数规划的代码实现。

整数规划模型如下所示。

$$\max\ z = 100x_1 + 150x_2 \tag{10.50}$$

$$\text{s.t.}\quad 2x_1 + x_2 \leqslant 10 \tag{10.51}$$

$$3x_1 + 6x_2 \leqslant 40 \tag{10.52}$$

$$x_1, x_2 \geqslant 0 \tag{10.53}$$

$$x_1, x_2 \in \mathbb{Z} \tag{10.54}$$

下面给出分支定界算法求解上述整数规划模型的 Python 代码（调用 Gurobi）。为了方便读者理解，我们首先给出下面的详细伪代码。该伪代码与前面章节介绍的伪代码大体相同，只是细节上略有改动。

Algorithm 5 分支定界算法 3

初始化：根据 (10.50)~(10.54) 初始化模型，创建根节点 S，且设置其对应的模型为 IP 的线性松弛
设置 UB $\leftarrow \infty$, LB $\leftarrow 0$
设置节点集合 $Q \leftarrow \{S\}$ 和当前最优解 $x^* \leftarrow$ Null
while Q 非空且 UB $-$ LB $> \epsilon$ **do**
    选择当前节点 $S^i \leftarrow Q$.pop() (对应的模型为 P^i)
    status $\leftarrow$ 求解模型 P^i 的线性松弛模型
    更新对偶界限 $z^i \leftarrow$ 线性松弛模型的目标函数
    x^i(LP) $\leftarrow$ 线性松弛模型的解
    if status 不是最优 **then**
        根据不可行性剪枝
    else if $z^i \leqslant$ LB **then**
        根据界限剪枝
    else if x^i(LP) 是整数解 **then**
        更新原界限 LB $\leftarrow z^i$
        更新当前最优解 $x^* \leftarrow x^i$(LP)
        根据最优性剪枝

```
    else if status 是最优且 x^i(LP) 是小数解 then
        将当前解 x^i(LP) 圆整为整数解 x^i_int(LP)
        if 圆整后整数解的目标函数 x^i_int(LP) > LB then
            更新 LB ← x^i_int(LP) 对应的目标函数
            更新当前最优解 x* ← x^i_int(LP)
        end if
        选择 x^i(LP) 中第一个小数变量为分支变量
        创建两个子节点 S^i_1 和 S^i_2, 其对应的模型分别为 P^i_1 和 P^i_2
        更新节点的集合 Q ← Q ∪ {S^i_1, S^2_1}
        tempUB ← 剩余叶子节点集合 Q 中节点的线性松弛模型的目标函数的最大值
        更新 UB ← tempUB
    end if
end while
return 最优解 x*
```

在上述伪代码中，为了快速得到一些可行解，我们在每个小数解的节点执行了向下圆整的操作，向下圆整操作对于该模型来讲一定可以得到一个整数可行解（注意，该操作对其他模型不一定适用），这对更新 LB 很有帮助。另外，为了更快地更新 UB，我们在分支操作后求解了所有叶子节点的线性松弛模型。

Branch and Bound

```
# Liu Xinglu
# hsinglul@163.com
# Tsinghua University
# 2021-3-11

# Branch and Bound Algorithm
# max 100x1 + 150x2
# 2x1 + x2 ⩽ 10
# 3x1 + 6x2 ⩽ 40
# x1, x2 ⩾ 0, and integer

# Creat LP
from gurobipy import *
import numpy as np
import copy
import matplotlib.pyplot as plt

RLP = Model('relaxed MIP')
x = {}
```

```
for i in range(2):
    x[i] = RLP.addVar(lb = 0
                      ,ub = GRB.INFINITY
                      ,vtype = GRB.CONTINUOUS
                      ,name = 'x_' + str(i)
                      )

RLP.setObjective(100 * x[0] + 150 * x[1], GRB.MAXIMIZE)

RLP.addConstr(2 * x[0] + x[1] <= 10, name = 'c_1')
RLP.addConstr(3 * x[0] + 6 * x[1] <= 40, name = 'c_2')

RLP.optimize()

# Node class
class Node:
    # this class defines the node
    def __init__(self):
        self.local_LB = 0
        self.local_UB = np.inf
        self.x_sol = {}
        self.x_int_sol = {}
        self.branch_var_list = []
        self.model = None
        self.cnt = None
        self.is_integer = False

    def deepcopy_node(node):
        new_node = Node()
        new_node.local_LB = 0
        new_node.local_UB = np.inf
        new_node.x_sol = copy.deepcopy(node.x_sol)
        new_node.x_int_sol = copy.deepcopy(node.x_int_sol)
        new_node.branch_var_list = []
        new_node.model = node.model.copy()
        new_node.cnt = node.cnt
        new_node.is_integer = node.is_integer

        return new_node

# Branch and Bound
def Branch_and_bound(RLP):
```

```
    # initialize the initial node
    RLP.optimize()
    global_UB = RLP.ObjVal
    global_LB = 0
    eps = 1e-3
    incumbent_node = None
    Gap = np.inf

    '''
        Branch and Bound starts
    '''
    # creat initial node
    Queue = []
    node = Node()
    node.local_LB = 0
    node.local_UB = global_UB
    node.model = RLP.copy()
    node.model.setParam("OutputFlag", 0)
    node.cnt = 0
    Queue.append(node)

    cnt = 0
    Global_UB_change = []
    Global_LB_change = []
    while (len(Queue) > 0 and global_UB - global_LB > eps):
        # select the current node
        current_node = Queue.pop()
        cnt += 1

        # solve the current model
        current_node.model.optimize()
        Solution_status = current_node.model.Status

        '''
        OPTIMAL = 2
        INFEASIBLE = 3
        UNBOUNDED = 5
        '''

        # check whether the current solution is integer and execute prune
        # step
        '''
```

```
        is_integer : mark whether the current solution is integer solution
        Is_Pruned : mark whether the current solution is pruned
        '''
        is_integer = True
        Is_Pruned = False
        if (Solution_status == 2):
            for var in current_node.model.getVars():
                current_node.x_sol[var.varName] = var.x
                print(var.VarName, ' = ', var.x)

                # # round the solution to get an integer solution
                current_node.x_int_sol[var.varName] = (int)(var.x)  # round
# the solution to get an integer solution
                if (abs((int)(var.x) - var.x) >= eps):
                    is_integer = False
                    current_node.branch_var_list.append(var.VarName)  # to
# record the candidate branch variables

            # update the LB and UB
            if (is_integer == True):
                # For integer solution node, update the LB and UB
                current_node.is_integer = True
                current_node.local_LB = current_node.model.ObjVal
                current_node.local_UB = current_node.model.ObjVal
                if (current_node.local_LB > global_LB):
                    global_LB = current_node.local_LB
                    incumbent_node = Node.deepcopy_node(current_node)
            if (is_integer == False):
                # For integer solution node, update the LB and UB also
                current_node.is_integer = False
                current_node.local_UB = current_node.model.ObjVal
                current_node.local_LB = 0
                for var_name in current_node.x_int_sol.keys():
                    var = current_node.model.getVarByName(var_name)
                    current_node.local_LB+=current_node.x_int_sol[var_name]
* var.Obj
                if (current_node.local_LB > global_LB or (
                        current_node.local_LB == global_LB and current_node.
is_integer == True)):
                    global_LB = current_node.local_LB
                    incumbent_node = Node.deepcopy_node(current_node)
                    incumbent_node.local_LB = current_node.local_LB
```

```
                incumbent_node.local_UB = current_node.local_UB

        '''
            PRUNE step
        '''
        # prune by optimility
        if (is_integer == True):
            Is_Pruned = True

        # prune by bound
        if (is_integer == False and current_node.local_UB < global_LB):
            Is_Pruned = True

        Gap = round(100 * (global_UB - global_LB) / global_LB, 2)
        print('\n ------------ \n', cnt, '\t Gap  = ', Gap, '  %')
    elif (Solution_status != 2):
        # the current node is infeasible or unbound
        is_integer = False
        '''
            PRUNE step
        '''
        # prune by infeasiblity
        Is_Pruned = True
        continue

    '''
      BRANCH step
    '''
    if (Is_Pruned == False):
        # selecte the branch variable
        branch_var_name = current_node.branch_var_list[0]
        left_var_bound = (int)(current_node.x_sol[branch_var_name])
        right_var_bound = (int)(current_node.x_sol[branch_var_name]) + 1

        # creat two child nodes
        left_node = Node.deepcopy_node(current_node)
        right_node = Node.deepcopy_node(current_node)

        # creat left child node
        temp_var = left_node.model.getVarByName(branch_var_name)
        left_node.model.addConstr(temp_var <= left_var_bound, name=
    'branch_left_' + str(cnt))
```

```
            left_node.model.setParam("OutputFlag", 0)
            left_node.model.update()
            cnt += 1
            left_node.cnt = cnt

            # creat right child node
            temp_var = right_node.model.getVarByName(branch_var_name)
            right_node.model.addConstr(temp_var >= right_var_bound, name=
        'branch_right_' + str(cnt))
            right_node.model.setParam("OutputFlag", 0)
            right_node.model.update()
            cnt += 1
            right_node.cnt = cnt

            Queue.append(left_node)
            Queue.append(right_node)

            # update the global UB, explor all the leaf nodes
            temp_global_UB = 0
            for node in Queue:
                node.model.optimize()
                if(node.model.status == 2):
                    if(node.model.ObjVal >= temp_global_UB):
                        temp_global_UB = node.model.ObjVal


            global_UB = temp_global_UB
            Global_UB_change.append(global_UB)
            Global_LB_change.append(global_LB)

    # all the nodes are explored, update the LB and UB
    global_UB = global_LB
    Gap = round(100 * (global_UB - global_LB) / global_LB, 2)
    Global_UB_change.append(global_UB)
    Global_LB_change.append(global_LB)

    print('\n\n\n\n')
    print('----------------------------------------')
    print('         Branch and Bound terminates     ')
    print('         Optimal solution found          ')
    print('----------------------------------------')
    print('\nFinal Gap  = ', Gap, '  %')
```

```
    print('Optimal Solution:', incumbent_node.x_int_sol)
    print('Optimal Obj:', global_LB)

    return incumbent_node, Gap, Global_UB_change, Global_LB_change


# Solve the IP model by branch and bound
incumbent_node, Gap, Global_UB_change, Global_LB_change = Branch_and_bound
    (RLP)


# plot the results
# fig = plt.figure(1)
# plt.figure(figsize=(15,10))
font_dict = {"family":'Arial',    #"Kaiti",
      "style":"oblique",
       "weight":"normal",
       "color":"green",
       "size": 20
      }

plt.rcParams['figure.figsize'] = (12.0, 8.0) # 单位是inches
plt.rcParams["font.family"] = 'Arial' #"SimHei"
plt.rcParams["font.size"] = 16

x_cor = range(1, len(Global_LB_change) + 1)
plt.plot(x_cor, Global_LB_change, label = 'LB')
plt.plot(x_cor, Global_UB_change, label = 'UB')
plt.legend()
plt.xlabel('Iteration', fontdict=font_dict)
plt.ylabel('Bounds update', fontdict=font_dict)
plt.title('Bounds update during branch and bound procedure \n', fontsize =
    23)
plt.savefig('Bound_updates.eps')
plt.show()
```

算法迭代过程中，Bounds 的更新过程如图 10.16 所示。

10.9 Python 调用 Gurobi 实现分支定界算法求解 TSP

本节尝试使用分支定界算法求解 TSP。基本的分支定界算法求解 TSP 的效率并不高。为了方便，我们仍然用 Solomon VRP benchmark 中的数据作为 TSP 的测试数据。

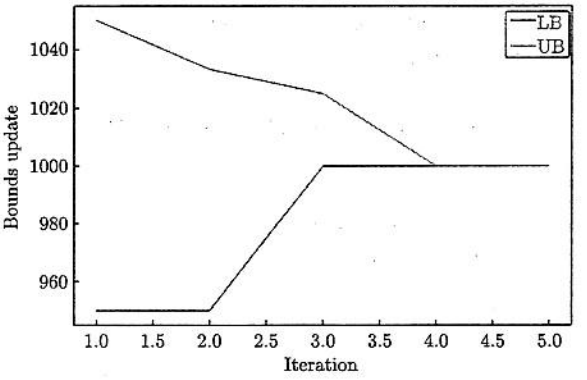
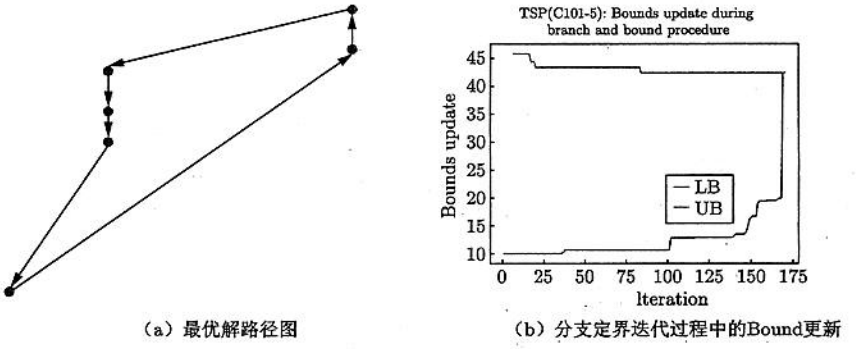


图 10.16 Bounds 更新过程

选取 C101 的前 5 个客户点为测试数据，加上仓库点，共 6 个点。求解过程中的 UB 和 LB 的更新如图 10.17 所示。



(a) 最优解路径图　　(b) 分支定界迭代过程中的Bound更新

图 10.17 6 个点的 TSP 算例测试结果

当客户点为 10 个及以上时，求解就比较困难了。因此用分支定界算法求解 TSP 效率并不高。不过本节的主要目的是学习分支定界算法本身，而不是如何提升求解效率。感兴趣的读者可以继续深入探索如何加速求解。关于加快分支定界算法求解 TSP 的速度，一个比较可行的加速方法就是利用拉格朗日松弛提供较好的下界，具体介绍见第 12 章。

10.10 Python 调用 Gurobi 实现分支定界算法求解 VRPTW

本节尝试用分支定界算法求解 VRPTW。本节代码使用深度优先对 BB tree 进行搜索。在每个小数解的节点，选择取值最接近 0.5 的变量 x_{ij} 进行分支（详细代码见附配资源）。

我们选取 Solomon VRP benchmark 中的 C101 的前 10 个点作为算例进行测试，设置车辆数为 2。分支定界算法迭代过程中，UB 和 LB 的迭代如图 10.18 所示。

由图 10.18 可知，最基本的分支定界算法求解 VRPTW 是非常困难的，即使是 10 个

客户点、2 辆车的小算例，迭代次数也要将近 12 万次，求解时间为 17 分钟左右。此外，算法迭代过程中，LB 更新非常缓慢。

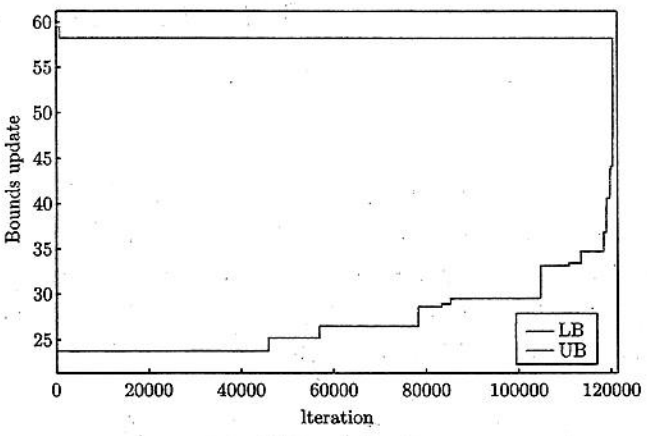


图 10.18 VRPTW：算例 C101-10 的 Bounds 更新过程

为了加速求解，在建模之前，可以先对网络拓扑图进行预处理，删除容量、时间窗明显不满足要求的弧。经过预处理之后，我们发现，基本的分支定界算法求解 VRPTW 的效率有显著的提升。就 C101 而言，提取前 25、前 50 个客户点作为测试算例，求解都比较快。对于 100 个客户点的算例，也可以在几小时之内得到最优解。其中 50 个点的算例迭代过程中的 UB 和 LB 更新如图 10.19 所示（设置车辆数为 6），求解时间仅为 4.3 秒。求解较快的原因是，C 类算例容易求解，预处理之后变量个数明显减少。对于 C101-50，原本的 0-1 变量为 $52 \times 51 \times 6 = 15912$ 个，经过预处理之后，缩减为 7308 个。

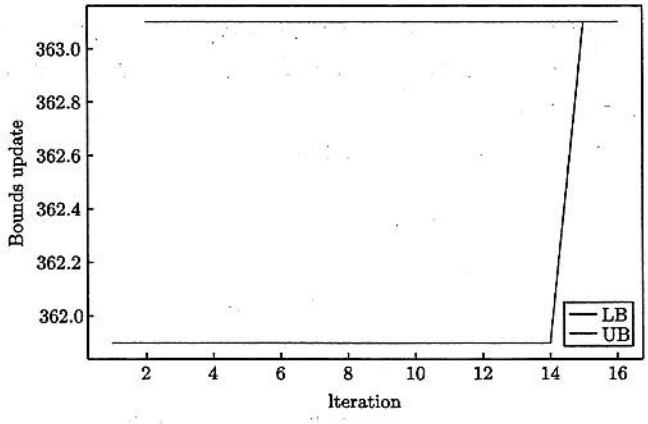


图 10.19 VRPTW: 算例 C101-50 的 Bounds 更新过程

第11章　分支切割算法

分支切割算法（Branch and Cut Algorithm）是非常强大的求解混合整数规划的精确算法，本书涉及的两款求解器 Gurobi 和 CPLEX 的主体算法框架都是分支切割算法。该算法由 Padberg、Manfred、Rinaldi 和 Giovanni 于 1991 年提出，用于求解大规模 STSP 问题（Padberg and Rinaldi，1991）。目前为止，分支切割算法是求解一般的混合整数规划（即不依赖具体问题特性的混合整数规划）问题的最有效的精确算法。

11.1　什么是分支切割算法

之前我们介绍了分支定界算法，分支定界算法的主要思想是将根节点的线性松弛问题的可行域通过切割，分成小的子区域，然后一步一步搜寻获得整数规划的最优解。但是仅仅使用分支定界算法，在很多情况下是不够的，我们需要加入其他技术来加速算法，提高算法的性能。本节要讲述的分支切割算法就是一种非常强大的算法。分支切割算法也是目前流行的优秀求解器采用的算法框架，例如 Gurobi、CPLEX 等求解器，核心框架都是分支切割算法。

分支切割算法就是在分支定界算法的基础上，在分支的同时，根据节点的解的信息以及其他有用信息（例如一些问题特性等），给节点对应的子问题构造并添加割平面（Cutting plane），在缩小子问题的可行域的同时，又不排除任何可行整数解，从而更快地找到最优解的一种高效的算法。也正是由于加入的割平面没有割去任何可行整数解，所以分支切割算法也是可以保证最优性的。

我们来看下面的例子（Winston and Goldberg, 2004）：

$$\max\ z = 8x_1 + 5x_2 \tag{11.1}$$

$$\text{s.t.}\quad x_1 + x_2 \leqslant 6 \tag{11.2}$$

$$9x_1 + 5x_2 \leqslant 45 \tag{11.3}$$

$$x_1, x_2 \geqslant 0, x_1, x_2 \in \mathbb{Z} \tag{11.4}$$

其线性松弛问题的可行域如图 11.1 所示，线性松弛问题的最优解为 $x_1 = 3.75, x_2 = 2.25$，$z_{\text{LP}} = 41.25$，而整数规划的最优解为 $x_1 = 5, x_2 = 0, z^* = 40$。

如果我们用分支定界算法求解该问题，第一步首先选取分数部分最大的 x_1 进行分支，将其分成 $x_1 \leqslant 3$ 和 $x_1 \geqslant 4$ 两个子区域，如图 11.2 所示。

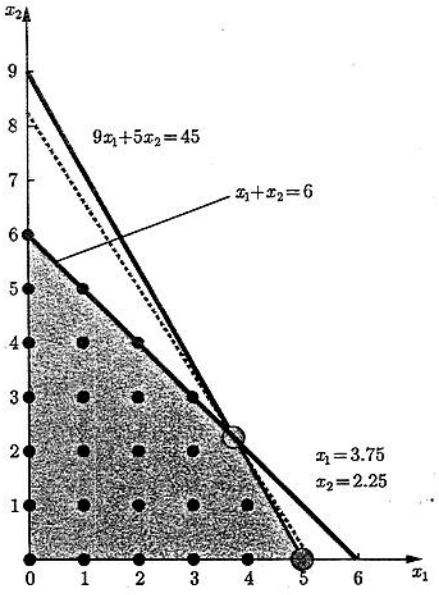


图 11.1 线性松弛的可行域

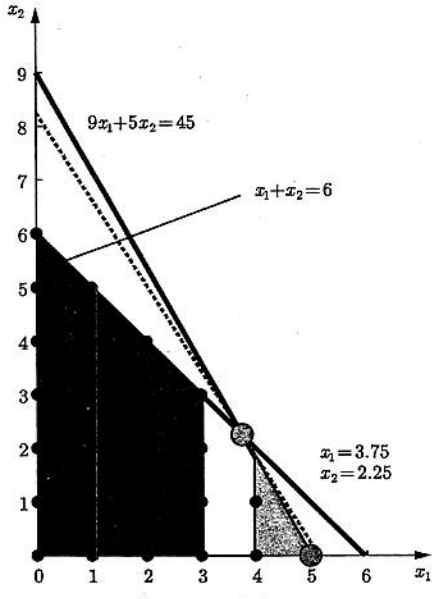


图 11.2 分支 x_1

在这里，分支后我们发现右边的区域包含着最优解，但是并不能一次性找出来，此时如果我们把这个子区域切一刀，只保留包含几个整数解的区域，把不包含整数可行解的部分切掉。如图 11.3 所示，我们在右边的子区域添加一条割平面

$$3x_1 + 2x_2 \leqslant 15$$

这里，我们并没有割掉任何整数解，但是目标函数 $z = 8x_1 + 5x_2$ 却可以直接在右边区域对应的子问题 11.1.5 中，得到最优整数解 $x_1 = 5, x_2 = 0, z^* = 40$，从而使得该子节点变成已查明，而整个问题的全局 LB 被更新为 40，当前的最好解（current best）也被更新为该解。

$$\max\ z = 8x_1 + 5x_2 \tag{11.5}$$

$$\text{s.t.}\quad x_1 + x_2 \leqslant 6 \tag{11.6}$$

$$9x_1 + 5x_2 \leqslant 45 \tag{11.7}$$

$$x_2 \geqslant 4 \qquad \text{分支约束} \tag{11.8}$$

$$\boxed{3x_1 + 2x_2 \leqslant 15} \qquad \text{割平面} \tag{11.9}$$

$$x_1, x_2 \geqslant 0 \tag{11.10}$$

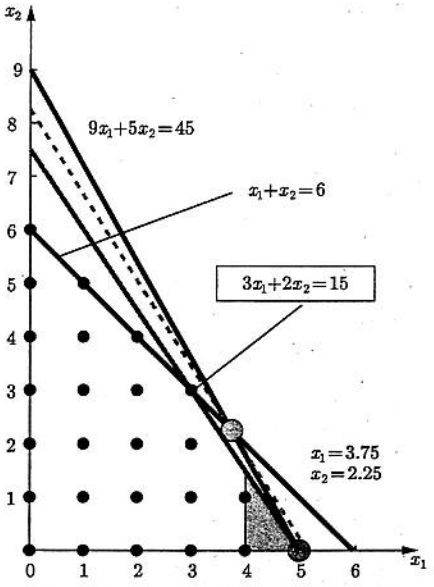


图 11.3　在右分支中添加一个割平面

这无疑使得整个算法搜索空间显著减少，加速了算法。这种在执行分支定界算法的过程中同时添加除了分支约束以外的其他约束，以辅助缩小子问题的可行域，又不剔除任何原问题的可行整数解的方法，就叫作分支切割算法。显然，分支定界算法是分支切割算法的一个特例，也就是分支的时候在子节点只添加一条分支约束（也就是割平面）。

在讲解分支切割算法之前，我们首先来介绍一个重要的概念：**有效不等式**（Valid Inequality）。

11.2 有效不等式

首先，我们考虑下面的整数规划（IP）：

$$\text{IP} \qquad \max\{\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} : \boldsymbol{x} \in X\} \tag{11.11}$$

其中，$X = \{\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b}, \boldsymbol{x} \in \mathbb{Z}_+^n\}$。

这里的符号 $X = \{\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b}, \boldsymbol{x} \in \mathbb{Z}_+^n\}$ 可能对于一些初学者有些陌生。这样的写法是为了简略，省去了烦琐的约束表达式，而直接用 X 表示该整数规划的可行域，X 就是该整数规划的一系列约束构成的可行域。而 $\boldsymbol{x} \in X$ 则是该可行域内的任意可行解。

定理 11.2.1 （Wolsey，1998）IP 的所有整数可行解构成的可行域（离散的点）的凸包 $\mathbf{Conv}(X) = \{\boldsymbol{x} : \widetilde{\boldsymbol{A}}\boldsymbol{x} \leqslant \widetilde{\boldsymbol{b}}, \boldsymbol{x} \geqslant \boldsymbol{0}\}$ 是一个多面体。

其中，$\mathbf{Conv}(X)$ 表示可行域 X 的凸包（Convex Hull）。$\widetilde{\boldsymbol{A}}\boldsymbol{x} \leqslant \widetilde{\boldsymbol{b}}$ 精确地刻画出了可行域 X 的凸包。

举一个非常简单的例子，对于整数规划 (11.1)，其可行域如下：

$$x_1 + x_2 \leqslant 6 \tag{11.12}$$

$$9x_1 + 5x_2 \leqslant 45 \tag{11.13}$$

$$x_1, x_2 \geqslant 0 \tag{11.14}$$

$$x_1, x_2 \in \mathbb{Z} \tag{11.15}$$

这些约束的可行域为一些离散的点。如图 11.4（a）中的黑点，而该图中对应的解集的凸包的一个精确刻画可以为

$$x_1 + x_2 \leqslant 6 \tag{11.16}$$

$$9x_1 + 5x_2 \leqslant 45 \tag{11.17}$$

$$-x_1 \leqslant 0 \tag{11.18}$$

$$-x_2 \leqslant 0 \tag{11.19}$$

$$\boxed{3x_1 + 2x_2 \leqslant 15} \tag{11.20}$$

因此有

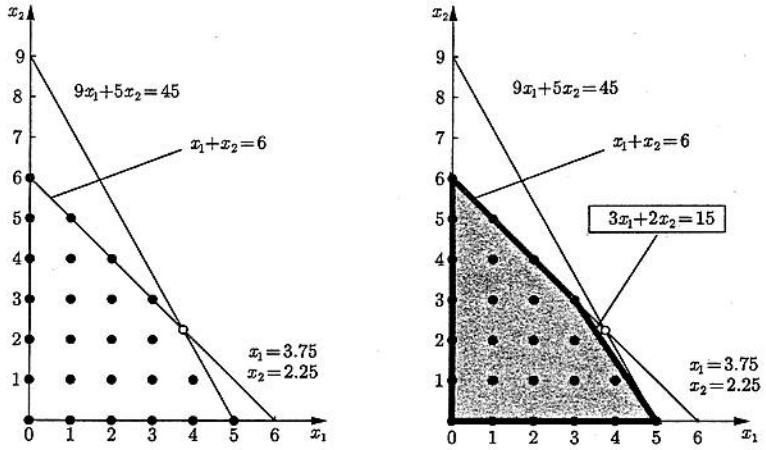
$$\widetilde{\boldsymbol{A}} = \begin{bmatrix} 1 & 1 \\ 9 & 5 \\ -1 & 0 \\ 0 & -1 \\ 3 & 2 \end{bmatrix}, \widetilde{\boldsymbol{b}} = \begin{bmatrix} 6 \\ 45 \\ 0 \\ 0 \\ 15 \end{bmatrix}$$

因此 $\widetilde{\boldsymbol{A}}\boldsymbol{x} \leqslant \widetilde{\boldsymbol{b}}$ 就是整数规划 (11.1) 的凸包，是对其可行域的凸包的一个刻画，如图 11.4（b）所示。

对于 IP 来讲，最优整数解同样也一定在凸包中，因此在可行域内搜索最优解和在凸包内搜索最优解是等价的。根据这一点，我们可以将原问题重新建模（Reformulation）成下面的线性规划：

$$\text{LP} \quad \max\{c^{\mathrm{T}}x : \tilde{A}x \leqslant \tilde{b}, x \geqslant 0\} \tag{11.21}$$

并且，对于所有的 c（也就是任意线性目标函数），LP 的最优极点就是 IP 的最优解。对于混合整数规划，该结论也成立。



（a）整数规划的可行域　　（b）可行域的凸包

图 11.4　整数规划可行域的凸包

有了上面的结论，我们很容易就可以有一个很大胆的想法。既然凸包对应的 LP 的最优极点就是 IP 的最优解，那我们只要找到 IP 的所有可行解的凸包，然后求解一个线性规划就可以完美地解决整数规划了。

对于最小生成树问题和指派问题来讲，凸包的刻画比较容易，这两类问题都不是 NP-hard 问题。但是对于 NP-hard 问题，找到它们的凸包的清晰刻画基本是不可能的。因此，只能采取一些办法去逼近它们的凸包。

因此本章介绍的有效不等式（Valid Inequality）以及割平面，就是来完成这一任务的。它们的功能就是不断地切割可行域，不断地逼近凸包。

为了寻找凸包，需要将原来问题的约束进行加强。或者说，在原有的约束中，有些约束不够紧，使得可行域距离凸包很远。而我们可以将其转化，生成一些比较紧的、更靠近凸包的约束，以更好地逼近凸包。这就是有效不等式的作用。

根据之前的描述得知，要想逼近凸包，就需要尽可能地将线性松弛问题的不包含任何整数可行解的部分切掉。这表明很多时候当前的约束并不够紧。可以通过改写约束，使得约束变紧，但整数规划的解依然不变。比如下面的例子：

$$X = \{(x, y) : x \leqslant 100y, 0 \leqslant x \leqslant 10, y = 0 \text{ or } 1\}$$

可以很容易得到上述问题更紧的一个版本，比如我们可以添加约束

$$x \leqslant 10y$$

这里，$x \leqslant 10y$ 就是 X 的一个有效不等式（Valid Inequality）。之后将有效不等式用 $\pi^{\mathrm{T}}x \leqslant \pi_0$ 的形式来表示，例如 $3x_1 + 2x_2 \leqslant 15$ 这样的形式。

下面给出有效不等式的定义。

定义 11.2.1 （Wolsey，1998）任给 $x \in X$，不等式 $\pi^{\mathrm{T}}x \leqslant \pi_0$ 都成立，则不等式 $\pi^{\mathrm{T}}x \leqslant \pi_0$ 是 $X \subseteq \mathbb{R}^n$ 的有效不等式。

如果有整数规划的可行域 $X = \{x \in \mathbb{Z}^n : \boldsymbol{A}x \leqslant b\}$ 和其凸包 $\mathbf{Conv}(X) = \{x \in \mathbb{R}^n : \widetilde{\boldsymbol{A}}x \leqslant \widetilde{b}\}$，不难得出，约束 $a^i x \leqslant b_i$ 和 $\widetilde{a}^i x \leqslant \widetilde{b}^i$ 就是 X 的有效不等式。

关于有效不等式的更多详细解释，参见文献（Wolsey，1998）。这里仅给出一个定理，该定理在之后介绍 Gomory 割平面时也会用到。

定理 11.2.2 （Wolsey，1998）设有 $X = \{y \in \mathbb{Z}^1 : y \leqslant b\}$，则不等式 $y \leqslant \lfloor b \rfloor$ 是 X 的一个有效不等式。

在详细介绍分支切割算法之前，我们首先来介绍割平面算法（Cutting Plane Algorithms）。割平面算法是除了分支定界算法以外，求解整数规划的又一个著名的经典算法。

11.3 割平面算法

割平面算法有很多种，本节仅对基本的割平面进行详细阐述。为了便于读者更好地理解割平面算法，我们首先给出割平面算法的一个通用框架。

假设初始模型为 $P = \{x : \boldsymbol{A}x \leqslant b, x \geqslant \mathbf{0}\}$，且 $X = P \cup \mathbb{Z}^n$。（注意，这里的符号可能会引起困惑，P 指的是线性规划的可行域；$\mathbb{Z}^n$ 指的是整数约束。$P \cup \mathbb{Z}^n$ 就表示整数规划的可行域。）

假设有 $X = P \cup \mathbb{Z}^n$ 和 X 的一系列的有效不等式族 $\mathcal{F} : \pi^{\mathrm{T}}x \leqslant \pi_0, (\pi, \pi_0) \in \mathcal{F}$。通常 $\mathcal{F}$ 会包含非常多的不等式（2^n 或者更多）。我们不是为了真正寻找到精确的凸包，而是想要尽可能地逼近它。

下面先来给出一个整数规划 IP : $\max\{c^{\mathrm{T}}x : x \in X\}$ 的基本的割平面算法，同时，假定已经生成了该 IP 的一个有效不等式族 $\mathcal{F}$。下面就是一个比较简洁的割平面算法的伪代码。（注：该伪代码就是循环找出一个问题的有效割平面，并添加到原模型中，使得原模型更紧。并不能保证一定得到整数最优解。）

Algorithm 6 割平面算法

1: 初始化：设置 $t \leftarrow 0$，$P^0 \leftarrow P$，最大迭代次数 maxCnt
2: **while** 未找到整数最优解或 $t <$ maxCnt **do**
3: 　求解 LP : $z^t \leftarrow \max\{cx : x \in P^t\}$
4: 　令 x^t 为 LP 的一个最优解
5: 　**if** $x^t \in \mathbb{Z}^n$ **then**
6: 　　停止，x^t 是 IP 的一个最优解，**return** x^t

```
    else if x^t ∉ Z^n then
        求解 x^t 和不等式族 F 构成的分割问题，即寻找割平面
    end if
    if 存在有效不等式 (π^t, π_0^t) ∈ F，且 π^t x^t > π_0^t 割去小数最优解 x^t then
        P^{t+1} ← P^t ∩ {x : π^t x ⩽ π_0^t}
        t ← t + 1
    else
        停止 and return P^t
    end if
end while
return P^t
```

如果上述算法结束时，并没有找到 IP 的一个整数解，则模型

$$P^t = P \cap \{\boldsymbol{x} : (\boldsymbol{\pi}^{\mathrm{T}})^i \boldsymbol{x} \leqslant \pi_0^i, \forall i = 1, 2, \cdots, t\} \tag{11.22}$$

就是 IP 的一个改进模型，约束更紧。可以将 P^t 放入分支定界算法中再继续求解。

注意：在每一次迭代中，不是只能添加一条割，而是可以添加很多条割。

至此就介绍完了上述通用框架，下面具体地介绍一些割平面算法。由于割平面算法有很多种，而且有一些割平面算法涉及很复杂的数学推导，讲解比较困难。为了读者更容易理解割平面算法，我们首先以最基础的 Gomory 割平面算法作为开始（Gomory, 1958）。

11.3.1 Gomory's 分数割平面算法

考虑下面的整数规划：

$$\max\{\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}, \boldsymbol{x} \geqslant \boldsymbol{0}, \boldsymbol{x} \in \mathbb{Z}^n\}$$

该方法的基本思路是，首先求解上述问题的线性松弛问题，得到线性松弛问题的最优基。如果该最优基对应的最优解不是整数解，用 Gomory 割的方法生成一条割，把这个非整数最优解从可行域中切掉。假设已经得到了最优基，该最优基对应的单纯形表可以写成下面的形式：

$$\max \quad \bar{a}_{00} + \sum_{j \in \mathrm{NB}} \bar{a}_{0j} x_j \tag{11.23}$$

$$\text{s.t.} \quad x_{B_u} + \sum_{j \in \mathrm{NB}} \bar{a}_{uj} x_j = \bar{a}_{u0}, \quad \forall u = 1, 2, \cdots, m \tag{11.24}$$

$$x \geqslant 0,\ x \in \mathbb{Z}^n \tag{11.25}$$

这里，m 是约束的个数，也是约束系数矩阵的行数。我们知道，最优基中的变量的个数是 m （暂时不考虑退化的情况），且 NB 指的是非基变量的集合，B 是基变量的集合。$\bar{a}_{0j} \leqslant 0, \forall j \in \mathrm{NB}$, 且 $\bar{a}_{u0} \geqslant 0, \forall u = 1, 2, \cdots, m$。这里 $\bar{a}_{u0}$ 其实就是这个 LP 线性松弛模型的最优解中决策变量 x_{Bu} 的具体取值，因为约束了 $x \geqslant 0$, 因此有 $\bar{a}_{u0} \geqslant 0$。